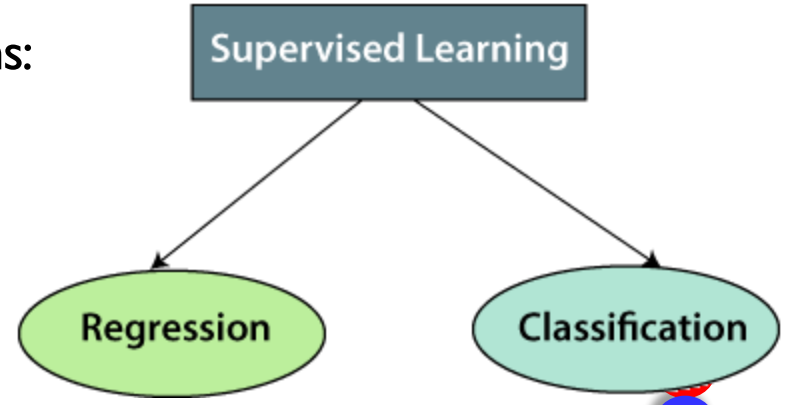
Supervised Learning

Supervised Machine Learning

- Supervised learning is the types of machine learning in which machines are trained using well "labelled" training data, and on basis of that data, machines predict the output.
- Supervised learning is a process of providing input data as well as correct output data to the machine learning model.
- The aim of a supervised learning algorithm is to **find a mapping function to map the input variable(x) with the output variable(y)**.

Types of supervised Machine learning Algorithms

- Supervised learning can be further divided into two types of problems:



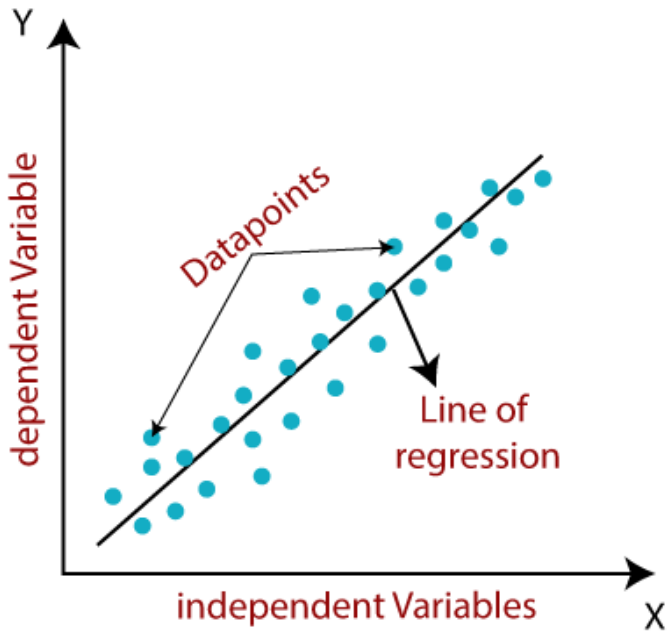
- Linear Regression in Machine Learning

is a statistical method that is used for predictive analysis.

- Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price,** etc.
- Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (y) variables

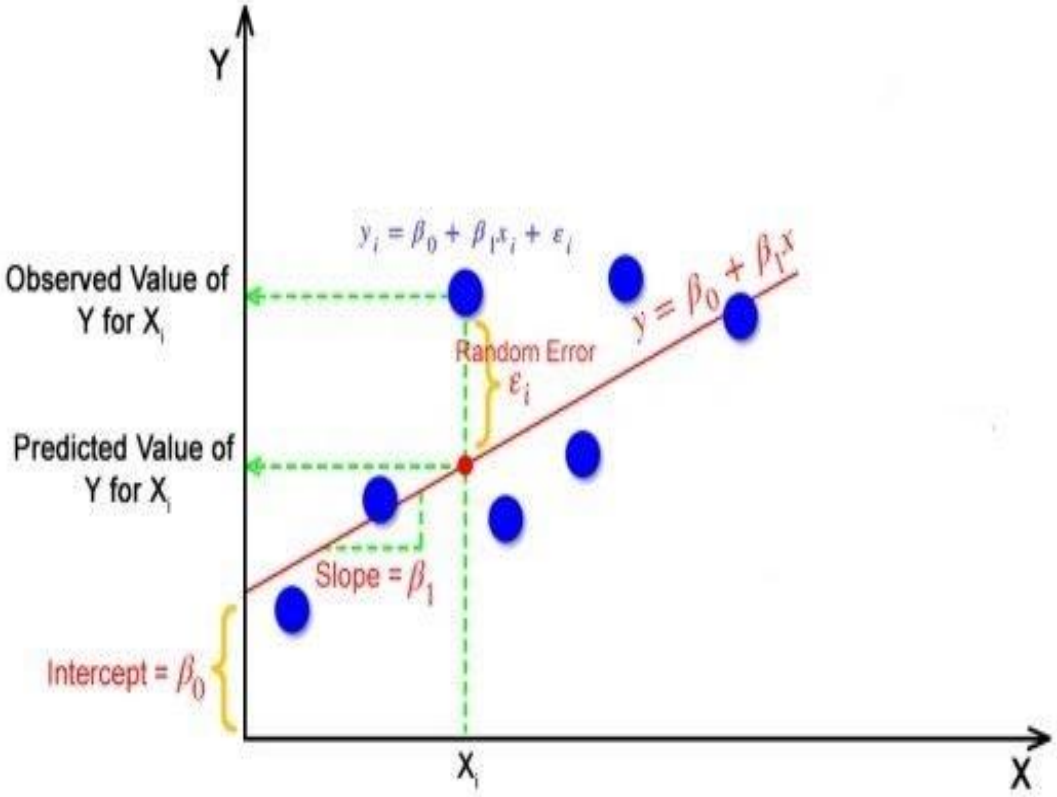
Linear Regression

- The linear regression model provides a sloped straight line representing the relationship between the variables.
- Mathematically, we can represent a linear regression as:
- **$Y_i = \beta_0 + \beta_1 X_i$**
- where Y_i = Dependent variable, β_0 = constant/Intercept, β_1 = Slope/Intercept, X_i = Independent variable.
- The goal of the linear regression algorithm is to get the **best values for B_0 and B_1** to find the best-fit line.



Linear Regression

- The best-fit line is a line that has the least error which means the error between predicted values and actual values should be minimum.
- In regression, the difference between the observed value of the dependent variable(**y i**) and the predicted value(**predicted**) is called the residuals.
- **ε i = y predicted – y i**
- **where y predicted = B 0 + B I X i**



Linear Regression

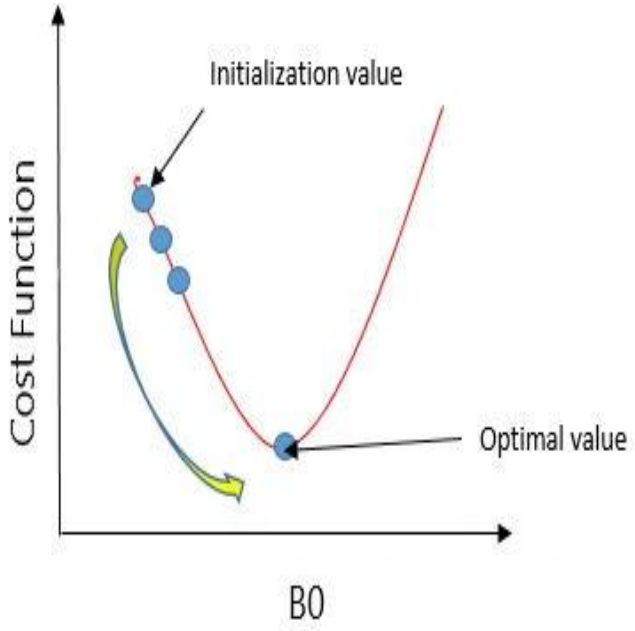
- The **cost function** helps to work out the optimal values for B 0 and B 1 , which provides the best-fit line for the data points.
- **Mean Squared Error (MSE)** cost function is used, which is the average squared error that occurred between the **y predicted** and **y i**.
- We calculate MSE using the simple linear equation y=mx+b:

$$MSE = \frac{1}{N}\sum_{i=1}^{n}(y_i - (B1x_i + B0))^2$$

- Using the MSE function, update the values of B 0 and B 1 such that the MSE value settles at the minima.
- These parameters can be determined using the gradient descent method such that the value for the cost function is minimum.

Linear Regression

- **Gradient Descent** is one of the optimization algorithms that optimize the cost function (objective function) to reach the optimal minimal solution
- To find the optimum solution, we need to reduce the cost function (MSE) for all data points.
- This is done by updating the values of the slope coefficient (B1) and the constant coefficient (B0) iteratively until we get an optimal solution for the linear function.
- A regression model uses gradient descent to update the coefficients of the line by reducing the cost function.
- It is done by a random selection of values of coefficient and then iteratively update the values to reach the minimum cost function.



Linear Regression

- Model Performance:
- The most used metrics are,
  - Coefficient of Determination or R-squared (R2)
  - Root Mean Squared Error (RSME) and Residual Standard Error (RSE)
- Coefficient of Determination or R-Squared (R2)
- R-squared is a number that explains the amount of variation that is explained/captured by the developed model. It always ranges between 0 & 1 . Overall, the higher the value of R-squared, the better the model fits the data.
- Mathematically it can be represented as,

$$R^2 = 1 - (RSS/TSS)$$

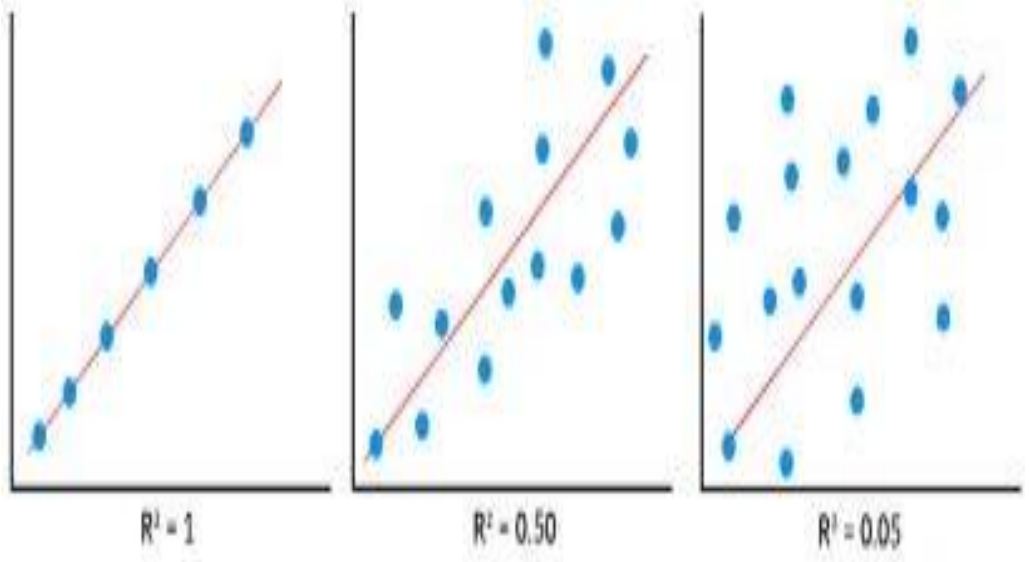
Linear Regression

- **Residual sum of Squares (RSS)** is defined as the sum of squares of the residual for each data point in the plot/data.
- It is the measure of the difference between the expected and the actual observed output.

$$\text{RSS} = \sum_{i=1}^{n} (y_i - b_0 - b_1 x_i)^2$$

- **Total Sum of Squares (TSS)** is defined as the sum of errors of the data points from the mean of the response variable.
- The significance of R-squared is shown by the following figures,

$$TSS = \sum (y_i - \bar{y}_i)^2$$



Linear Regression

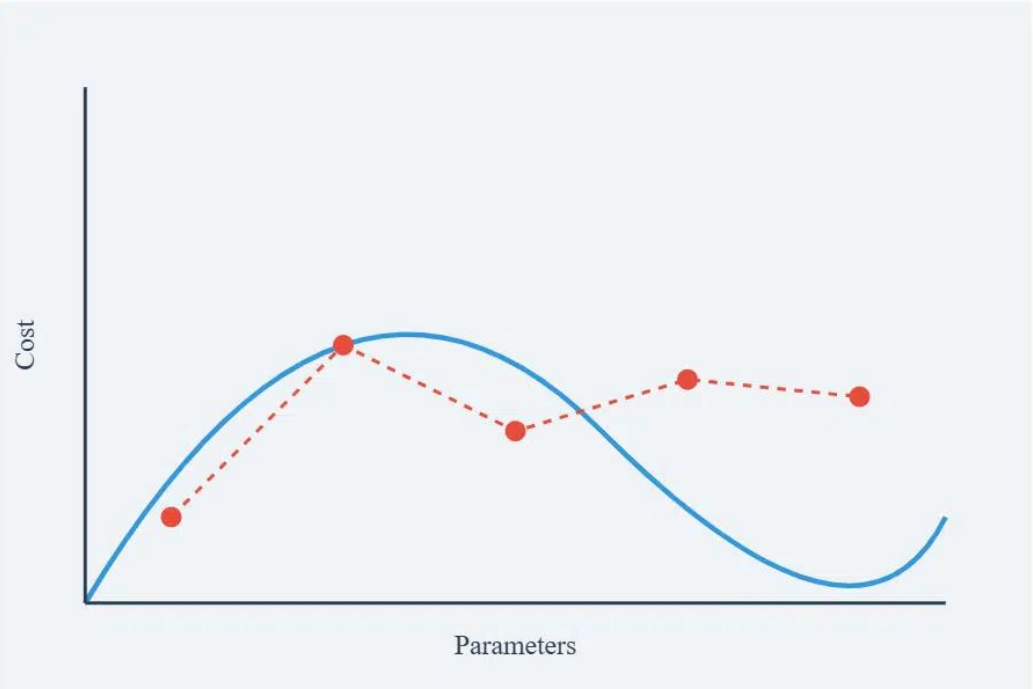
- **Root Mean Squared Error**
- The Root Mean Squared Error is the square root of the variance of the residuals.
- It specifies the absolute fit of the model to the data i.e. how close the observed data points are to the predicted values.
- To make this estimate unbiased, one has to divide the sum of the squared residuals by the **degrees of freedom** rather than the total number of data points in the model.
- This term is then called the **Residual Standard Error(RSE)**.

$$RMSE = \sqrt{\frac{RSS}{n}} = \sqrt{\sum_{i=1}^{n}\left(y_i^{Actual} - y_i^{Predicted}\right)^2 / n}$$

$$RSE = \sqrt{\frac{RSS}{df}} = \sqrt{\sum_{i=1}^{n}\left(y_i^{Actual} - y_i^{Predicted}\right)^2 / (n-2)}$$

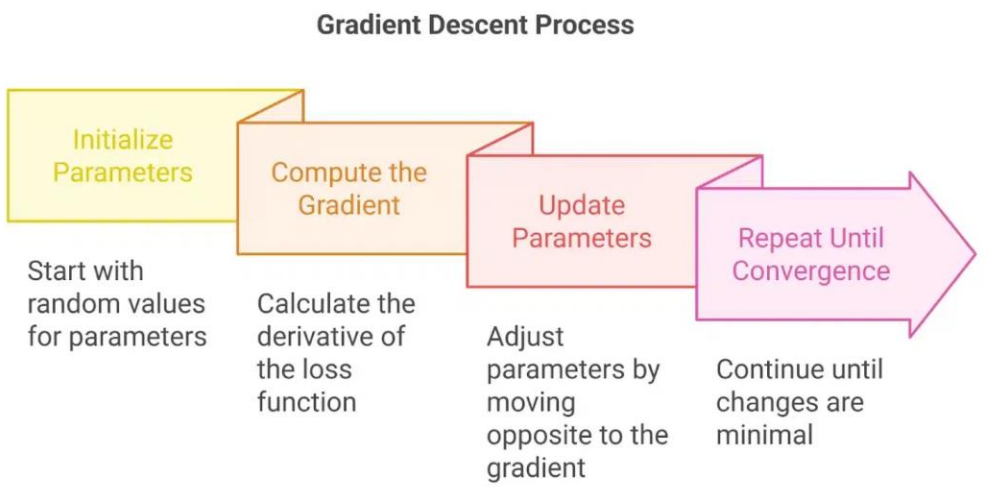
What is Gradient Descent?

- Gradient Descent is an optimization algorithm that helps machine learning models learn by updating parameters (such as weights in neural networks) to minimize the cost function.
- The cost function (or loss function) measures how well the model's predictions match the actual data. By iteratively adjusting the parameters in the direction that reduces the cost function, the model improves its accuracy.



How Gradient Descent Works?

- **Initialize Parameters** – Start with random values for the parameters (weights and biases).
- **Compute the Gradient** – Calculate the derivative (gradient) of the loss function with respect to each parameter.
- **Update Parameters** – Adjust the parameters by moving in the opposite direction of the gradient: Key points about the **formula $\theta = \theta - \alpha * \nabla J(\theta)$: Where:**
  θ represents model parameters
  α (learning rate) controls step size
  $\nabla J(\theta)$ is the gradient/slope of the cost function
- **Repeat Until Convergence** – Continue updating parameters until the change is minimal or a stopping criterion is met.



Gradient Descent

1. Linear Regression: Problem Setup

We assume a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{m}$$

For simplicity, start with **univariate linear regression**:

$$\hat{y}_i = h_\theta(x_i) = \theta_0 + \theta_1 x_i$$

2. Cost Function (Mean Squared Error)

Linear regression uses **MSE loss**:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2$$

Why the $\frac{1}{2}$?

- It cancels the 2 when differentiating
- Makes gradients cleaner

Gradient Descent (Batch Gradient Descent)

- **Idea :**Iteratively move parameters in the **negative direction of the gradient** to minimize the cost.

3. Partial derivative w.r.t. θ_0

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2m}\sum_{i=1}^{m} 2(\theta_0 + \theta_1 x_i - y_i)$$

Cancel 2:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m}\sum_{i=1}^{m} (\theta_0 + \theta_1 x_i - y_i)$$

4. Partial derivative w.r.t. θ_1

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{2m}\sum_{i=1}^{m} 2(\theta_0 + \theta_1 x_i - y_i)x_i$$

Cancel 2:

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m} (\theta_0 + \theta_1 x_i - y_i)x_i$$

Gradient Descent

5. Gradient Descent update rules (scalar)

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (\theta_0 + \theta_1 x_i - y_i)$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (\theta_0 + \theta_1 x_i - y_i) x_i$$

- **What is the Learning Rate (α)?**
- The **learning rate** controls **how big a step** we take in the direction of the negative gradient when updating parameters.
- Update rule (scalar form):

$$\theta := \theta - \alpha \frac{\partial J}{\partial \theta}$$

Gradient Descent

- For example, let's consider the below function as the cost function:

$f(x) = x^2 - 2x + 1$

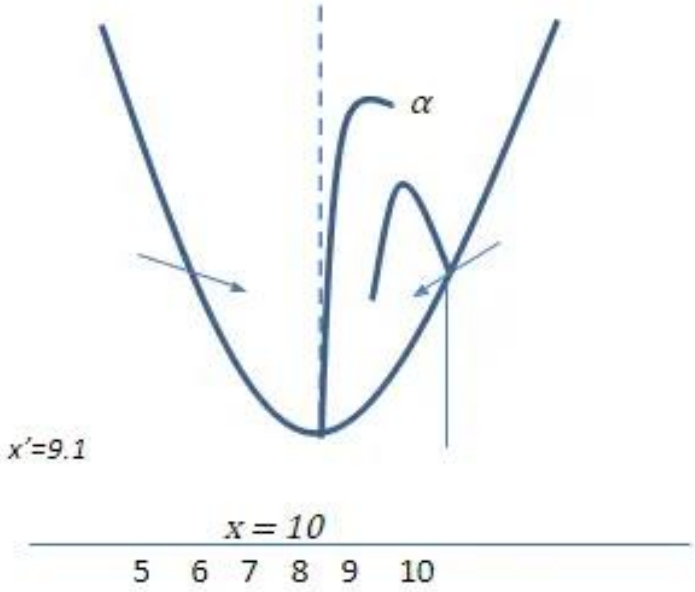
$x = 10$; *from the above plot*

Derivative of the function: $f'(x) = 2x - 2 = 18$

when x = 10;

x	f'	ϵ
10	18	0.05

$x' = 10 - (0.05 * 18) = 9.1$ → **New position of the x = 9.1**



Gradient Descent

Given the dataset:

x	y
1	2
2	3

Model:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Initial parameters:

$$\theta_0 = 0 \downarrow \theta_1 = 0$$

Step 1: Write the cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (\theta_0 + \theta_1 x_i - y_i)^2$$

Here $m = 2$

Gradient Descent

Step 2: Compute prediction errors

For $(x_1, y_1) = (1, 2)$:

$$h(1) = 0 + 0 = 0 \quad \Rightarrow \quad \text{error} = 0 - 2 = -2$$

For $(x_2, y_2) = (2, 3)$:

$$h(2) = 0 + 0 = 0 \quad \Rightarrow \quad \text{error} = 0 - 3 = -3$$

Step 3: Compute partial derivatives

For θ_0

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^{m} (\theta_0 + \theta_1 x_i - y_i)$$

$$= \frac{1}{2}(-2 + -3) = -2.5$$

Gradient Descent

For θ_1

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m}(\theta_0 + \theta_1 x_i - y_i)x_i$$

$$= \frac{1}{2}[(-2)(1) + (-3)(2)] = \frac{1}{2}(-2-6) = -4$$

Step 4: Update parameters

$$\theta_0 := \theta_0 - \alpha\frac{\partial J}{\partial \theta_0} = 0 - 0.1(-2.5) = 0.25$$

$$\theta_1 := \theta_1 - \alpha\frac{\partial J}{\partial \theta_1} = 0 - 0.1(-4) = 0.4$$

Final Answer (after 1 iteration)

$$\boxed{\theta_0 = 0.25, \quad \theta_1 = 0.4}$$

Stochastic Gradient Descent (SGD)

- Instead of using **all samples**, update parameters using **one sample at a time**.

Cost for a Single Sample

$$J_i(\theta) = \frac{1}{2}(h_\theta(x_i) - y_i)^2$$

Gradient for One Sample

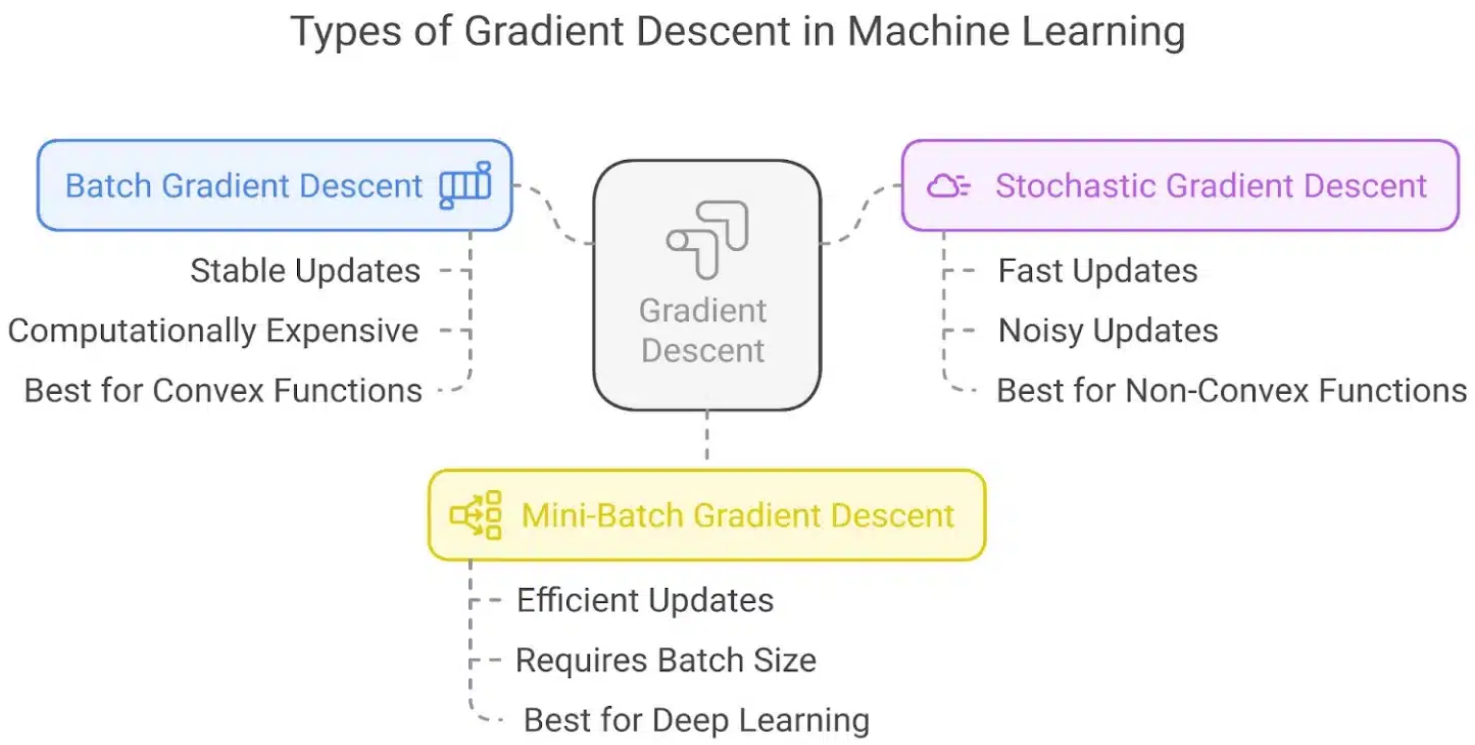
$$\nabla_\theta J_i(\theta) = (h_\theta(x_i) - y_i)x_i$$

SGD Update Rule

For each training example (x_i, y_i):

$$\theta := \theta - \alpha(h_\theta(x_i) - y_i)x_i$$

Types of Gradient Descent



Batch Gradient Descent (BGD)

- Batch Gradient Descent computes the gradient using the entire dataset before updating the model's parameters.
- **How it works**: The algorithm calculates the average gradient of the loss function over all training samples in one iteration.
- **Advantages**: Provides stable updates, leading to smooth convergence without oscillations.
- **Disadvantages**: Can be slow and computationally expensive for large datasets since it requires processing all data before each update.
- **Best suited for**: Convex cost functions where smooth updates are necessary, such as linear regression and logistic regression on small datasets.

Stochastic Gradient Descent (SGD)

- Stochastic Gradient Descent updates model parameters after computing the gradient for each individual training example.
- **How it works**: Instead of waiting for the entire dataset, SGD updates parameters immediately after evaluating a single training sample.
- **Advantages**: Faster than Batch Gradient Descent, especially for large datasets, and can help escape local minima.
- **Disadvantages**: Introduces more noise in updates, which can cause fluctuations in the optimization path.
- **Best suited for**: Non-convex functions where frequent updates can help explore different regions of the loss surface, such as training deep neural networks.

Stochastic Gradient Descent (SGD)

- Stochastic: "Process involving a randomly determined sequence of observations, each of which is considered as a sample of one element from a probability distribution." Or, in simple terms, "Random selection.
- **What is Optimization function?**
- Optimization is a mathematical technique of either minimizing or maximizing some function ***f(x)*** by altering x. In many real-time scenarios, we will be minimizing f(x). Well, if the thought comes up, how do we maximize the f(x)? It is just possible by minimizing the -f(x).
- **What is a Loss function?**
- Whenever we minimize our objective function ***f(x)*** we call it a loss function. Loss function takes various names such as ***Cost function or error function.***

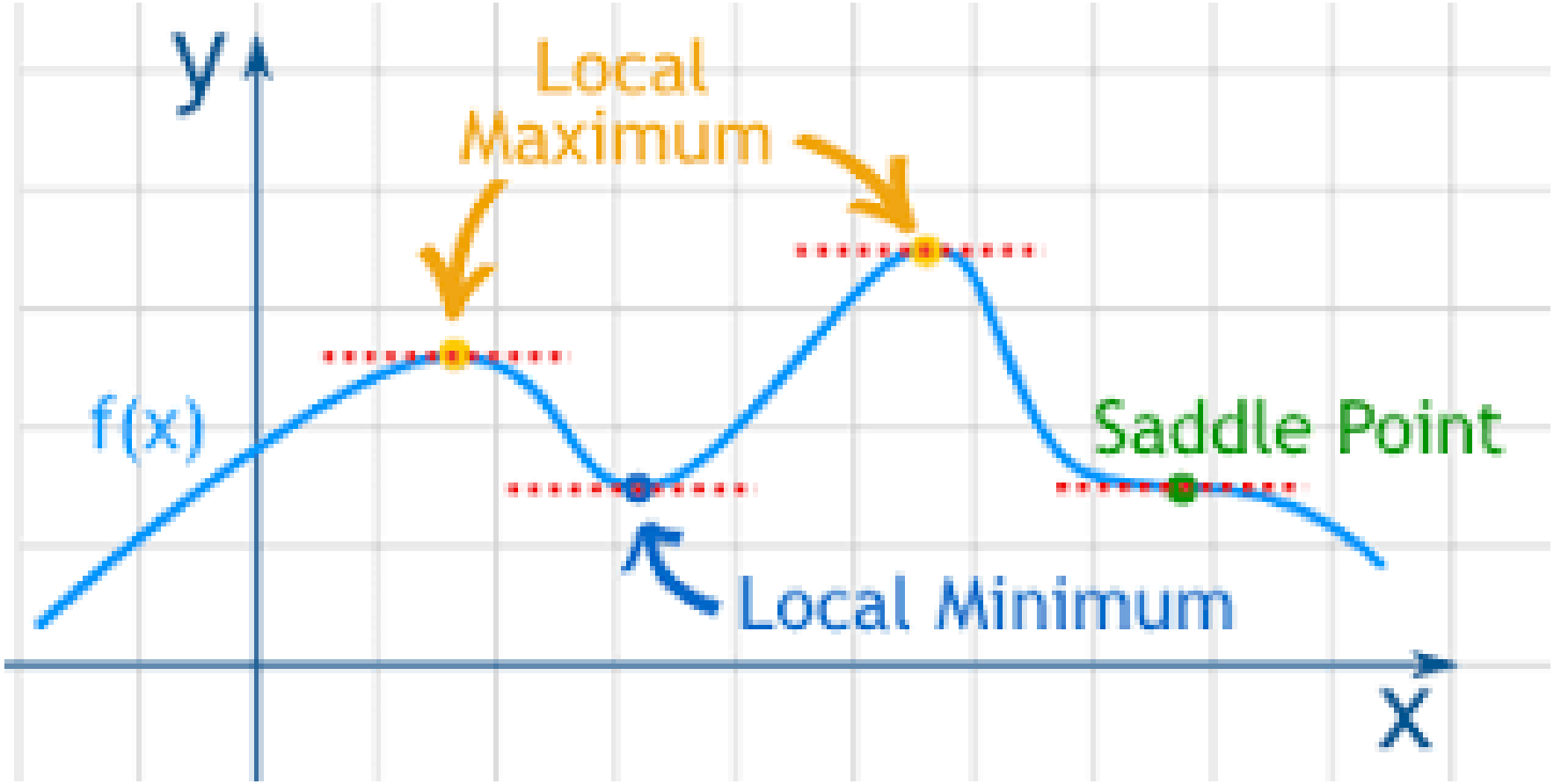
Gradient Descent

- **Local Minimum:**
- Local minimum is a point where f(x) is lower than all neighboring points, so it is no longer possible to decrease f(x) by making baby steps. This is also called as **local minima (or) relative minimum**.
- **Local Maximum:**
- Local Maximum is a point where f(x) is higher than at all neighboring points so it is not possible to increase f(x) by taking baby steps. This is also called a local **maxima (or) relative maximum.**
- **Saddle Points:**
- Some critical points or stationary points are neither maxima or minima, they are called Saddle points.

Gradient Descent

- **Global Minimum:**
- Global minimum is the smallest value of the function f(x) that is also called the **absolute minimum**. There would be only one global minimum whereas there could be more than one or more local minimum.
- When the input to the function is multidimensional most of the optimization functions fail to find the global minimum as they have multiple local minima, local maxima and saddle points.
- This is one of the greatest challenges in optimization, we often settle with the value of f that is minimal not necessarily global minimum in any formal sense.

Gradient Descent



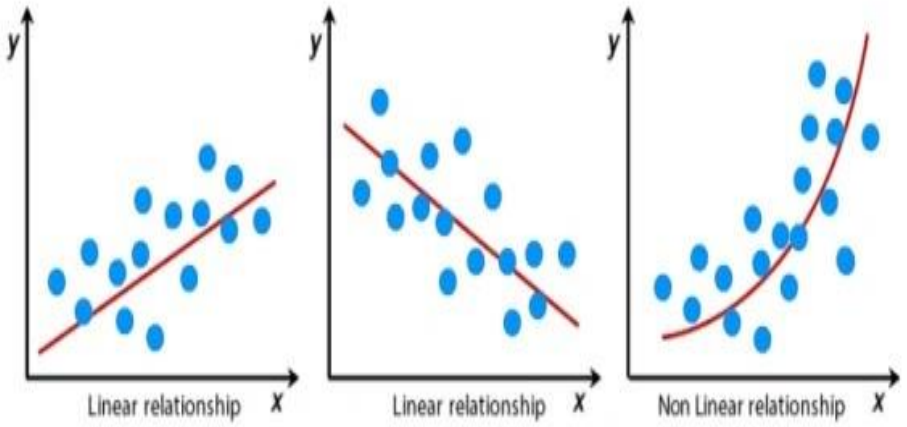
Mini-Batch Gradient Descent (MBGD)

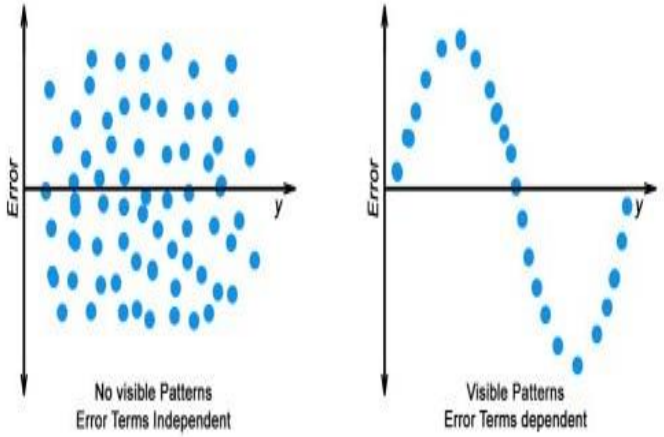
- Mini-Batch Gradient Descent finds a balance between BGD and SGD by computing gradients using a small batch of data.
- **How it works**: Instead of computing gradients for the entire dataset (BGD) or a single example (SGD), MBGD selects a small batch (e.g., 32 or 128 samples) and updates parameters accordingly.
- **Advantages**: More computationally efficient than BGD and more stable than SGD, reducing noise while speeding up convergence.
- **Disadvantages**: Requires selecting an appropriate batch size for optimal performance.
- **Best suited for**: Deep learning and large-scale machine learning applications where efficiency and stability are crucial.

Assumptions of Linear Regression

- **Linearity of residuals**: linear relationship between the dependent variable and independent variable(s).

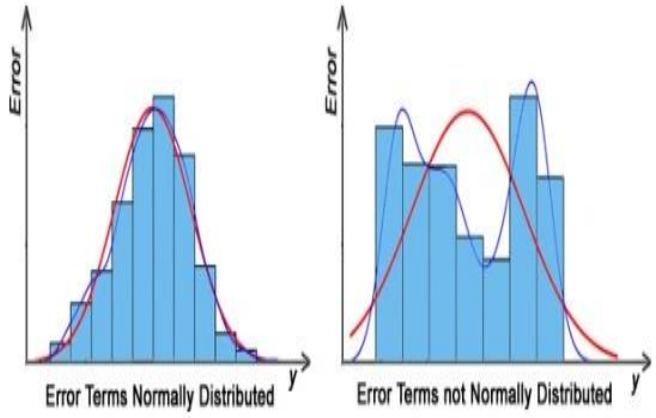
- **Independence of residuals:** The error terms should not be dependent on one another (like in time-series data wherein the next value is dependent on the previous one).

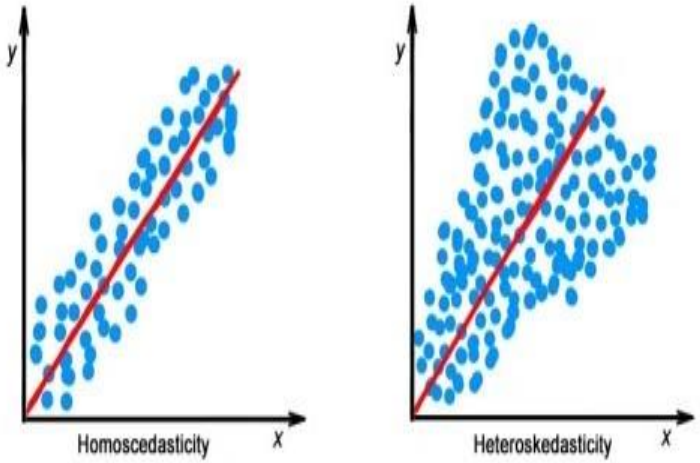




Assumptions of Linear Regression

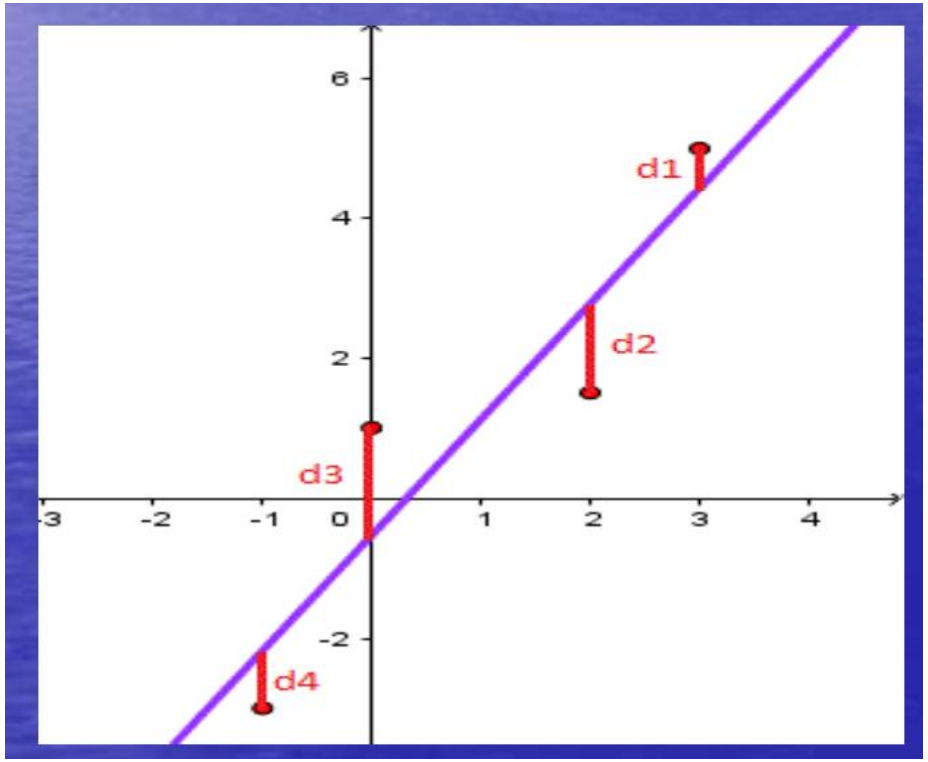
- **Normal distribution of residuals:** The mean of residuals should follow a **normal distribution** with a mean equal to zero or close to zero.
- **The equal variance of residuals:**
- The error terms must have constant variance. This phenomenon is known as **Homoscedasticity**. The presence of non-constant variance in the error terms is referred to as **Heteroscedasticity**.





Problems

- If the plot of n pairs of data (x , y) for an experiment appear to indicate a "linear relationship" between y and x, then the method of least squares may be used to write a linear relationship between x and y. The least squares regression line is the line that minimizes the sum of the squares (d1 + d2 + d3 + d4) of the vertical deviation from each data point to the line (see figure below as an example of 4 points)



- The least square regression line for the set of n data points is given by the equation of a line in slope intercept form:
- $y = a x + b$
- where a and b are given by

$$a = \frac{n\sum_{i=1}^{n} x_i y_i - \sum_{i=1}^{n} x_i \sum_{i=1}^{n} y_i}{n\sum_{i=1}^{n} x_i^2 - (\sum_{i=1}^{n} x_i)^2}$$

$$b = \frac{1}{n}(\sum_{i=1}^{n} y_i - a\sum_{i=1}^{n} x_i)$$

- Problem 1
- Consider the following set of points: {(-2 , -1) , (1 , 1) , (3 , 2)} a) Find the least square regression line for the given data points. b) Plot the given points and the regression line in the same rectangular system of axes.

Solutions

x	y	x y	x^2
-2	-1	2	4
1	1	1	1
3	2	6	9
$\Sigma x = 2$	$\Sigma y = 2$	$\Sigma xy = 9$	$\Sigma x^2 = 14$

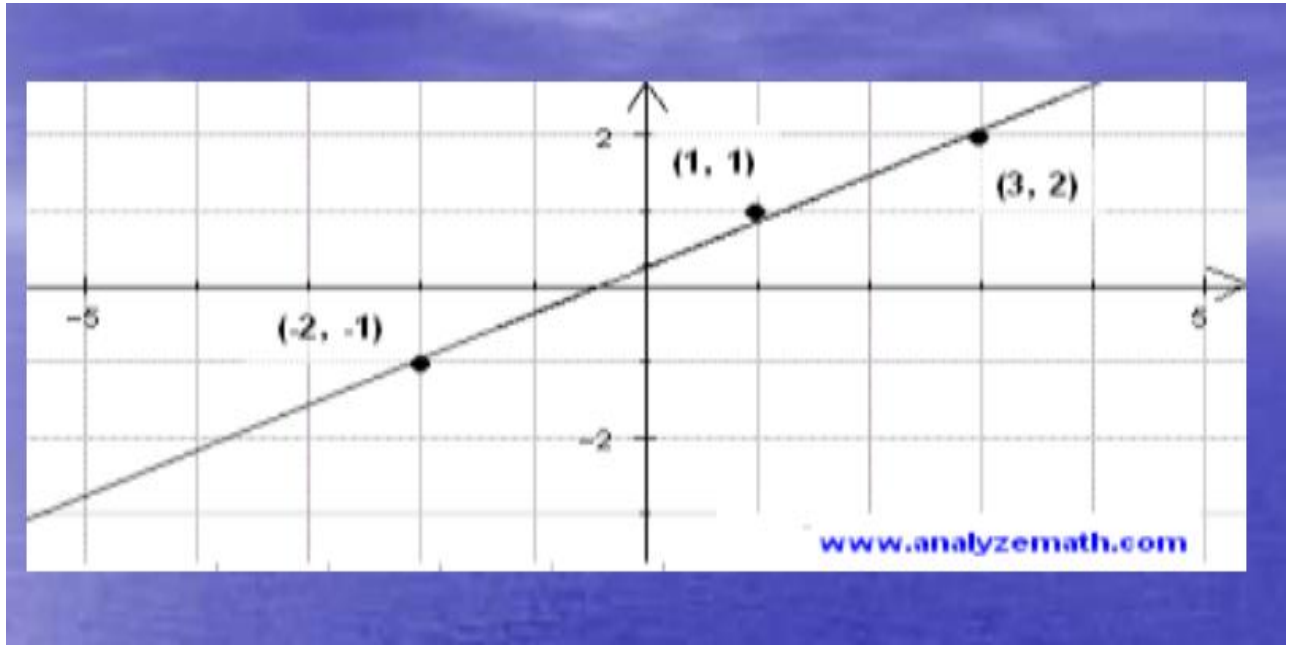
We now use the above formula to calculate a and b as follows

$a = (n\Sigma x\, y - \Sigma x \Sigma y) / (n\Sigma x^2 - (\Sigma x)^2) = (3*9 - 2*2) / (3*14 - 2^2) = 23/38$

$b = (1/n)(\Sigma y - a\ \Sigma x) = (1/3)(2 - (23/38)*2) = 5/19$

- We now graph the regression line given by

y = a x + b and the given points.



Problems

The values of y and their corresponding values of y are shown in the table below

x	0	1	2	3	4
y	2	3	5	4	6

a) Find the least square regression line $y = a x + b$.
b) Estimate the value of y when $x = 10$.

Solutions

x	y	x y	x^2
0	2	0	0
1	3	3	1
2	5	10	4
3	4	12	9
4	6	24	16
$\Sigma x = 10$	$\Sigma y = 20$	$\Sigma x\,y = 49$	$\Sigma x^2 = 30$

We now calculate a and b using the least square regression formulas for a and b.

$a = (n\Sigma x\,y - \Sigma x \Sigma y) / (n\Sigma x^2 - (\Sigma x)^2) = (5*49 - 10*20) / (5*30 - 10^2) = 0.9$

$b = (1/n)(\Sigma y - a\,\Sigma x) = (1/5)(20 - 0.9*10) = 2.2$

b) Now that we have the least square regression line $y = 0.9\,x + 2.2$, substitute x by 10 to find the value of the corresponding y.

$y = 0.9 * 10 + 2.2 = 11.2$

Simple Linear Regression

- import the three important libraries, which will help us for loading the dataset, plotting the graphs.

```
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

- load the dataset into our code:

```
data_set= pd.read_csv('Salary_Data.csv')
```

- extract the dependent and independent variables from the given dataset. The independent variable is years of experience, and the dependent variable is salary.

```
x= data_set.iloc[:, :-1].values  #  remove the last column
y= data_set.iloc[:, 1].values   #extract the second column
```

Simple Linear Regression

- split both variables into the test set and training set.

```
# Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 1/3, random_state=0)
```

- **Fitting the Simple Linear Regression to the Training Set:**

```
#Fitting the Simple Linear Regression model to the training dataset
from sklearn.linear_model import LinearRegression
regressor= LinearRegression()
regressor.fit(x_train, y_train)
```

Simple Linear Regression

- **Prediction of test set result:**

```
#Prediction of Test and Training set result
y_pred= regressor.predict(x_test)
x_pred= regressor.predict(x_train)
```

- **visualizing the Training set results:**

```
mtp.scatter(x_train, y_train, color="green")
mtp.plot(x_train, x_pred, color="red")
mtp.title("Salary vs Experience (Training Dataset)")
mtp.xlabel("Years of Experience")
mtp.ylabel("Salary(In Rupees)")
mtp.show()
```

Multiple Linear Regression

- **Multiple linear regression** is a technique to understand the relationship between a *single* dependent variable and *multiple* independent variables.
- The formulation for multiple linear regression is also similar to simple linear regression with the small change that instead of having one beta variable, you will now have betas for all the variables used. The formula is given as:
- $Y = B_0 + B_1X_1 + B_2X_2 + \ldots + B_pX_p + \varepsilon$

Multiple Linear Regression- Assumptions

- Considerations of Multiple Linear Regression
- All four assumptions made for Simple Linear Regression still hold for Multiple Linear Regression along with a few new additional assumptions.
- **Overfitting**: When more and more variables are added to a model, the model may become far too complex and usually ends up memorizing all the data points in the training set. This phenomenon is known as the overfitting of a model. This usually leads to high training accuracy and very low test accuracy.
- **Multicollinearity**: It is the phenomenon where a model with several independent variables, may have some variables interrelated.
- **Feature Selection:** With more variables present, selecting the optimal set of predictors from the pool of given features (many of which might be redundant) becomes an important task for building a relevant and better model.

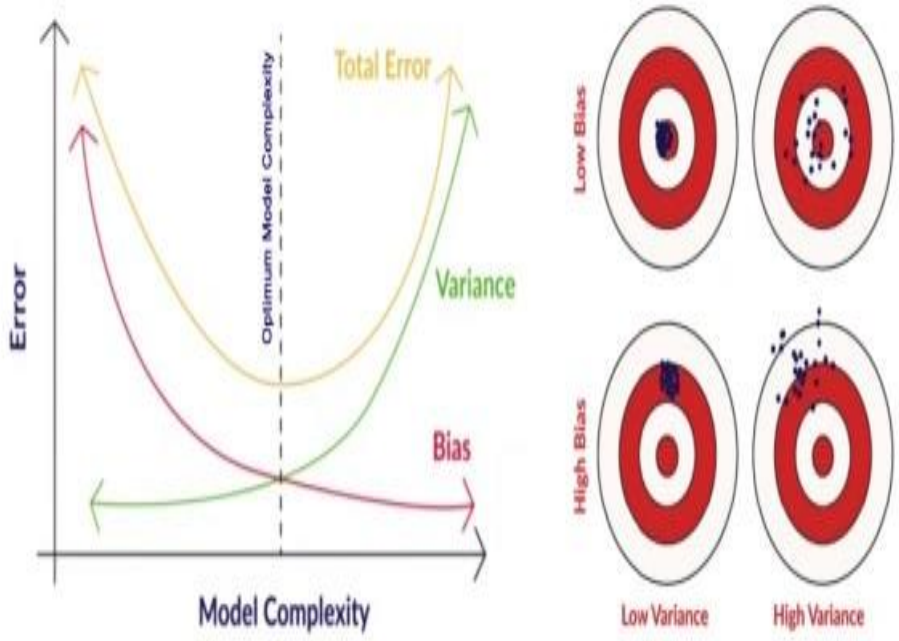
Multicollinearity

- Multicollinearity is a statistical phenomenon that occurs when two or more independent variables in a regression model are highly correlated, indicating a strong linear relationship among the predictor variables.
- As **multicollinearity** makes it difficult to find out which variable is contributing towards the prediction of the response variable.
- Multicollinearity can be detected using the following methods.
- **Pairwise Correlations:** Checking the pairwise correlations between different pairs of independent variables
- **Variance Inflation Factor (VIF)** VIF explains the relationship of one independent variable with all the other independent variables.VIF is given by,
- $Ri2$ = Coefficient of determination obtained when Xi is regressed on all the other independent variables. R2=1−sum of squares of error/ total sum of squares (from mean)
- If VIF > 10 then the value is high and it should be dropped

$$\text{VIF} = \frac{1}{1-R^2}$$

Overfitting and Underfitting in Linear Regression

- Bias
- Bias is a measure to determine how accurate a model's predictions are likely to be on future unseen data.
- linear algorithms have a high bias which makes them fast to learn and easier to understand but in general, are less flexible.
- Variance
- Variance is the sensitivity of the model towards training data, that is it quantifies how much the model will react when input data is changed.
- model shouldn't change too much from one training dataset to the next training data,



Overfitting

- When a model learns every pattern and noise in the data to such an extent that it affects the performance of the model on the unseen future dataset, it is referred to as ***overfitting***.
- The model fits the data so well that it interprets noise as patterns in the data.
- When a model has low bias and higher variance it ends up memorizing the data and causing overfitting.
- Overfitting causes the model to become specific rather than generic.

- The ways to prevent overfitting, which are stated below:

- Cross-validation
- If the training data is too small to train, add more relevant and clean data.
- If the training data is too large, do some feature selection and remove unnecessary features.

Underfitting

- When the model fails to learn from the training dataset and is also not able to generalize the test dataset, is referred to as ***underfitting***.
- When a model has high bias and low variance it ends up not generalizing the data and causing underfitting.
- It is unable to find the hidden underlying patterns in the data. This usually leads to low training accuracy and very low test accuracy.

- The ways to prevent underfitting are stated below,
- Increase the model complexity
- Increase the number of features in the training data
- Remove noise from the data.

Multiple Linear Regression

- Assumptions for Multiple Linear Regression:
- A **linear relationship** should exist between the Target and predictor variables.
- The regression residuals must be **normally distributed**.
- MLR assumes little or **no multicollinearity** (correlation between the independent variable) in data.
- **Step-1: Data Pre-processing Step:**

```
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

Multiple Linear Regression

- **importing dataset:** Now we will import the dataset(50_CompList), which contains all the variables. Below is the code for it:
- #importing datasets
- data_set= pd.read_csv('50_CompList.csv')
- **Extracting dependent and independent Variables:**
- #Extracting Independent and dependent Variable
- x= data_set.iloc[:, :-1].values
- y= data_set.iloc[:, 4].values

  array([[165349.2, 136897.8, 471784.1, 'New York'],
  [162597.7, 151377.59, 443898.53, 'California'],
  [153441.51, 101145.55, 407934.54, 'Florida'],

Multiple Linear Regression

- To encode the categorical variable into numbers, we will use the **LabelEncoder** class.
- #Catgorical data
- from sklearn.preprocessing **import** LabelEncoder, OneHotEncoder
- labelencoder_x= LabelEncoder()
- x[:, 3]= labelencoder_x.fit_transform(x[:,3])
- onehotencoder= OneHotEncoder(categorical_features= [3])
- x= onehotencoder.fit_transform(x).toarray()
- # Splitting the dataset into training and test set.
- from sklearn.model_selection **import** train_test_split
- x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.2, random_state=0)

Multiple Linear Regression

- #Fitting the MLR model to the training set:
- from sklearn.linear_model **import** LinearRegression
- regressor= LinearRegression()
- regressor.fit(x_train, y_train)
- Prediction of Test set results:
- #Predicting the Test set result;
- y_pred= regressor.predict(x_test)
- print('Train Score: ', regressor.score(x_train, y_train))
- print('Test Score: ', regressor.score(x_test, y_test))

Multiple Linear Regression by Hand

- Suppose we have the following dataset with one response variable *y* and two predictor variables X_1 and X_2:

- The estimated linear regression equation
- is: $\hat{y} = b_0 + b_1 * x_1 + b_2 * x_2$

y	X_1	X_2
140	60	22
155	62	25
159	67	24
179	70	20
192	71	15
200	72	14
212	75	14
215	78	11

Multiple Linear Regression by Hand

- **Step 1: Calculate X_1^2, X_2^2, X_1y, X_2y and X_1X_2.**

	y	X_1	X_2
	140	60	22
	155	62	25
	159	67	24
	179	70	20
	192	71	15
	200	72	14
	212	75	14
	215	78	11
Mean	181.5	69.375	18.125
Sum	1452	555	145

	X_1^2	X_2^2	X_1y	X_2y	X_1X_2
	3600	484	8400	3080	1320
	3844	625	9610	3875	1550
	4489	576	10653	3816	1608
	4900	400	12530	3580	1400
	5041	225	13632	2880	1065
	5184	196	14400	2800	1008
	5625	196	15900	2968	1050
	6084	121	16770	2365	858
Sum	38767	2823	101895	25364	9859

Multiple Linear Regression by Hand

- **Step 2: Calculate Regression Sums.**
- Next, make the following regression sum calculations:
- $\Sigma x_1^2 = \Sigma X_1^2 - (\Sigma X_1)^2 / n = 38{,}767 - (555)^2 / 8 =$ **263.875**
- $\Sigma x_2^2 = \Sigma X_2^2 - (\Sigma X_2)^2 / n = 2{,}823 - (145)^2 / 8 =$ **194.875**
- $\Sigma x_1 y = \Sigma X_1 y - (\Sigma X_1 \Sigma y) / n = 101{,}895 - (555*1{,}452) / 8 =$ **1,162.5**
- $\Sigma x_2 y = \Sigma X_2 y - (\Sigma X_2 \Sigma y) / n = 25{,}364 - (145*1{,}452) / 8 =$ **-953.5**
- $\Sigma x_1 x_2 = \Sigma X_1 X_2 - (\Sigma X_1 \Sigma X_2) / n = 9{,}859 - (555*145) / 8 =$ **-200.375**

Multiple Linear Regression by Hand

	y	X_1	X_2
	140	60	22
	155	62	25
	159	67	24
	179	70	20
	192	71	15
	200	72	14
	212	75	14
	215	78	11
Mean	181.5	69.375	18.125
Sum	1452	555	145

	X_1^2	X_2^2	X_1y	X_2y	X_1X_2
	3600	484	8400	3080	1320
	3844	625	9610	3875	1550
	4489	576	10653	3816	1608
	4900	400	12530	3580	1400
	5041	225	13632	2880	1065
	5184	196	14400	2800	1008
	5625	196	15900	2968	1050
	6084	121	16770	2365	858
Sum	38767	2823	101895	25364	9859
Reg Sums	263.875	194.875	1162.5	-953.5	-200.375

Multiple Linear Regression by Hand

- **Calculate b_0, b_1, and b_2.**
- The formula to calculate b_1 is: $[(\Sigma x_2^2)(\Sigma x_1 y) - (\Sigma x_1 x_2)(\Sigma x_2 y)] \ / \ [(\Sigma x_1^2)(\Sigma x_2^2) - (\Sigma x_1 x_2)^2]$
- Thus, $\mathbf{b_1} = [(194.875)(1162.5) - (-200.375)(-953.5)] \ / \ [(263.875)(194.875) - (-200.375)^2] = \mathbf{3.148}$
- The formula to calculate b_2 is: $[(\Sigma x_1^2)(\Sigma x_2 y) - (\Sigma x_1 x_2)(\Sigma x_1 y)] \ / \ [(\Sigma x_1^2)(\Sigma x_2^2) - (\Sigma x_1 x_2)^2]$
- Thus, $\mathbf{b_2} = [(263.875)(-953.5) - (-200.375)(1152.5)] \ / \ [(263.875)(194.875) - (-200.375)^2] = \mathbf{-1.656}$
- The formula to calculate b_0 is: $y - b_1 X_1 - b_2 X_2$
- Thus, $\mathbf{b_0} = 181.5 - 3.148(69.375) - (-1.656)(18.125) = \mathbf{-6.867}$

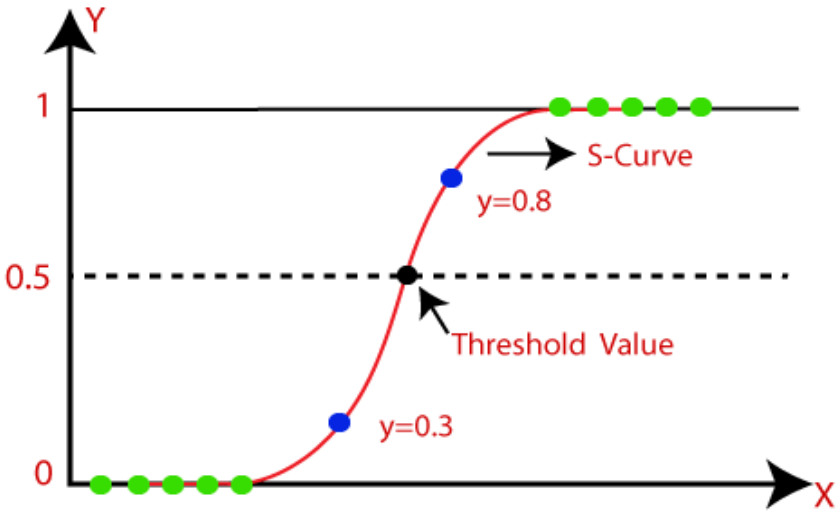
Multiple Linear Regression by Hand

- **Step 5: Place b_0, b_1, and b_2 in the estimated linear regression equation.**
- The estimated linear regression equation is: $\hat{y} = b_0 + b_1 * x_1 + b_2 * x_2$
- In our example, it is $\hat{y} = -6.867 + 3.148x_1 - 1.656x_2$

Logistic Regression

- It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc.
- **Logistic regression is used for solving the classification problems**.
- instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- It has the ability to provide probabilities and classify new data using continuous and discrete datasets.

Logistic Regression



- Logistic Function (Sigmoid Function):
- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form.
- The S-form curve is called the Sigmoid function or the logistic function.
- Assumptions for Logistic Regression:
  - The dependent variable must be categorical in nature.
  - The independent variable should not have multi-collinearity.

Logistic Regression

- Logistic Regression Equation:
- The Logistic regression equation can be obtained from the Linear Regression equation. The mathematical steps to get Logistic Regression equations are given below:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \cdots + b_nx_n$$

- In Logistic Regression y can be between 0 and 1 only, so for this let's divide the above equation by (1-y):

$$\frac{y}{1-y}$$; 0 for y= 0, and infinity for y=1

- But we need range between -[infinity] to +[infinity], then take logarithm

$$log\left[\frac{y}{1-y}\right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \cdots + b_nx_n$$

Logistic Regression

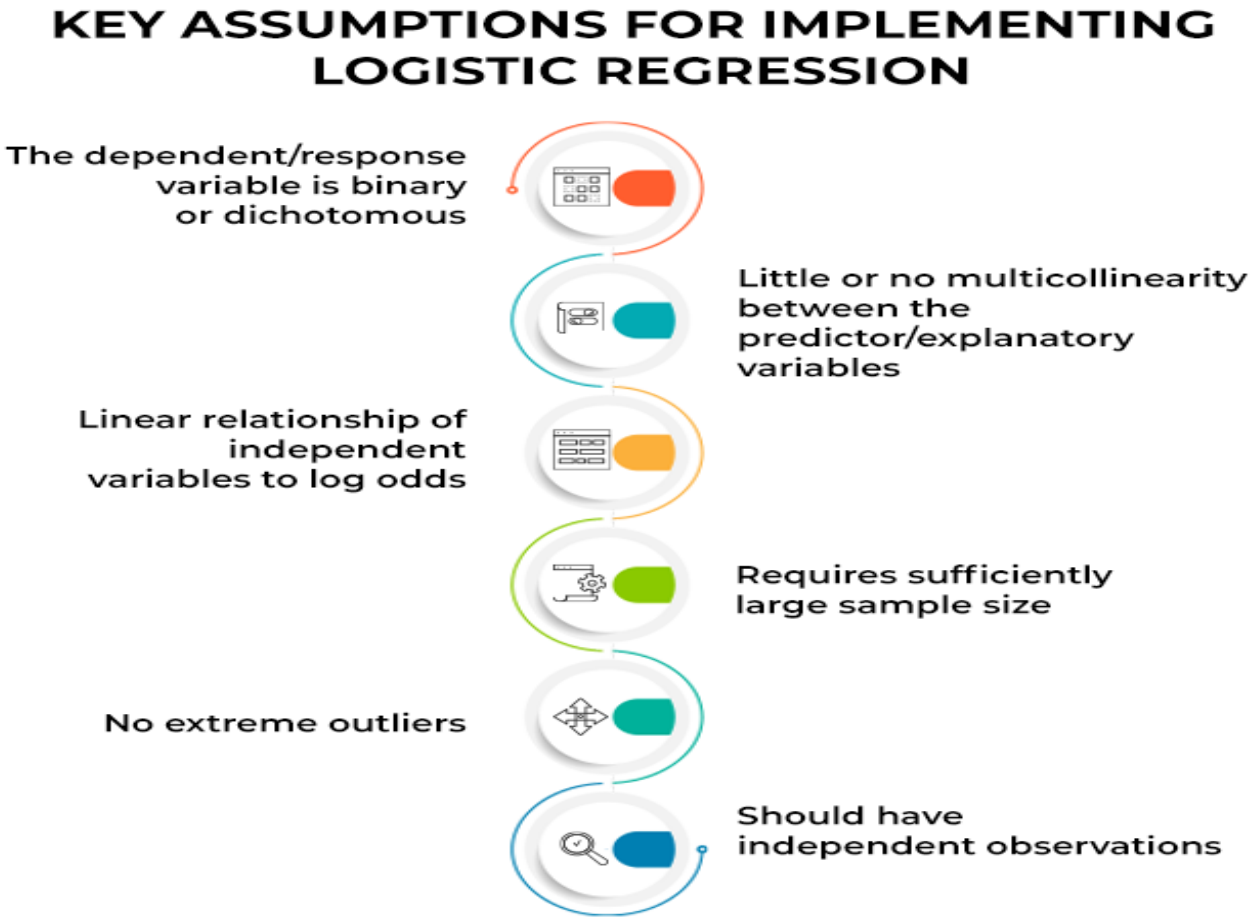
- Logistic regression uses a logistic function called a sigmoid function to map predictions and their probabilities.
- The sigmoid function is referred to as an activation function for logistic regression and is defined as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- where,
- e = base of natural logarithms
- value = numerical value one wishes to transform
- The following equation represents logistic regression:
- x = input value
- y = predicted output
- b0 = bias or intercept term
- b1 = coefficient for input (x)

$$y = \frac{e^{(b_0 + b_1 X)}}{1 + e^{(b_0 + b_1 X)}}$$

Key Assumptions



Logistic Regression

- Type of Logistic Regression:
- On the basis of the categories, Logistic Regression can be classified into three types:
- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"
- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".
- predict the **purchased variable (Dependent Variable)** by using **age and salary (Independent variables)**.

Logistic Regression

- **Data Pre-processing step:**

```
#Data Pre-procesing Step
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd

#importing datasets
data_set= pd.read_csv('user_data.csv')
#Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values
```

Logistic Regression

- # Splitting the dataset into training and test set.

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)
```

- #feature Scaling

```
from sklearn.preprocessing import StandardScaler
st_x= StandardScaler()
x_train= st_x.fit_transform(x_train)
x_test= st_x.transform(x_test)
```

- #Fitting Logistic Regression to the training set

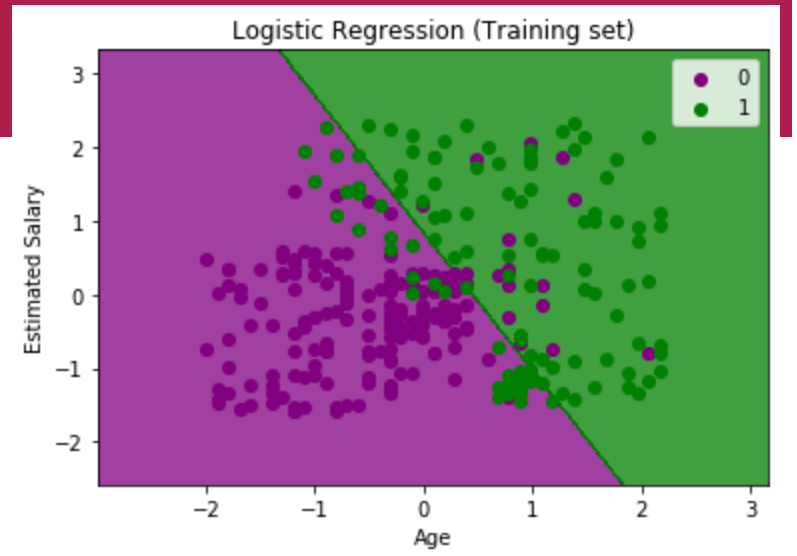
```
from sklearn.linear_model import LogisticRegression
classifier= LogisticRegression(random_state=0)
classifier.fit(x_train, y_train)
```

Logistic Regression

- **Predicting the Test Result**
- #Predicting the test set result
- y_pred= classifier.predict(x_test)
- **Test Accuracy of the result**
- #Creating the Confusion matrix
- from sklearn.metrics **import** confusion_matrix
- cm= confusion_matrix()
- **Visualizing the training set result**

Logistic Regression

- The **purple point observations** are for which purchased (dependent variable) is probably 0, i.e., users who did not purchase the SUV car.
- The **green point observations** are for which purchased (dependent variable) is probably 1 means user who purchased the SUV car.
- But there are some purple points in the green region (Buying the car) and some green points in the purple region(Not buying the car)



Question: Logistic Regression (Numerical Problem)

A logistic regression model is used to predict whether a customer will **purchase a product** **(1)** or **not (0)** based on the number of **hours spent on a website**.

The trained model is given by:

$$\log\left(\frac{p}{1-p}\right) = -8 + 0.6x$$

where

- x = hours spent on the website
- p = probability of purchase

a) Compute the probability that a customer who spends 12 hours on the website will make a purchase.
b) Determine the minimum number of hours a customer must spend on the website so that the probability of purchase is at least 80%.

(a) Probability of purchase for 12 hours

Step 1: Compute log-odds

$$z = -8 + 0.6(12) = -8 + 7.2 = -0.8$$

Step 2: Convert to probability

$$p = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{0.8}}$$

$$p = \frac{1}{1 + 2.225}$$

$$\boxed{p \approx 0.31}$$

Step 1: Substitute the required probability

$$\frac{1}{1+e^{-z}} = 0.8$$

Step 2: Multiply both sides by $(1+e^{-z})$

$$1 = 0.8(1+e^{-z})$$

Step 3: Expand the right-hand side

$$1 = 0.8 + 0.8e^{-z}$$

Step 4: Rearrange the equation

$$1 - 0.8 = 0.8e^{-z}$$

$$0.2 = 0.8e^{-z}$$

Step 5: Divide both sides by 0.8

$$e^{-z} = \frac{0.2}{0.8}$$

$$e^{-z} = 0.25$$

Step 6: Take natural logarithm on both sides

$$-z = \ln(0.25)$$

$$-z = -\ln(4)$$

Step 7: Multiply both sides by -1

$$\boxed{z = \ln(4) \approx 1.386}$$

Substitute into model

$$1.386 = -8 + 0.6x$$

$$0.6x = 9.386$$

$$\boxed{x \approx 15.64}$$

K-Nearest Neighbor(KNN) Algorithm

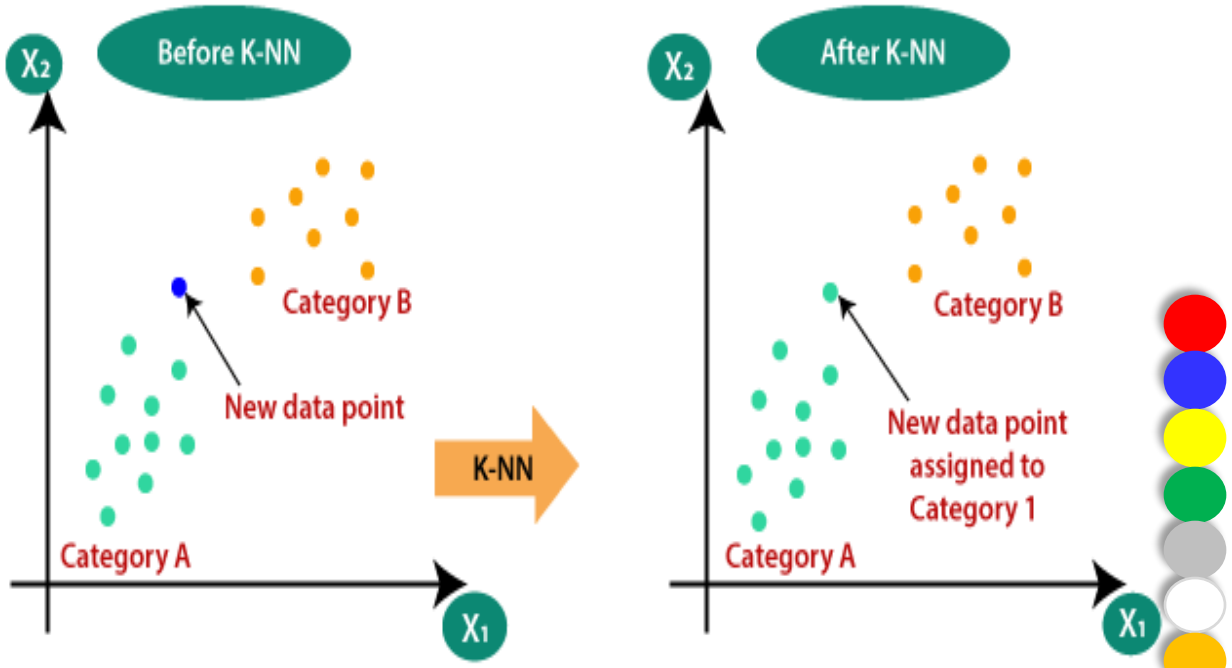
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm -** it does not learn from the training set immediately ,at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

K-Nearest Neighbor(KNN)

- image of a creature that looks similar to cat and dog,
- want to know either it is a cat or dog.
- KNN model will find the similar features of the new data set to the cats and dogs images a
- based on the most similar features it will put it in either cat or dog category.

Why do we need a K-NN Algorithm?

- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories.
- To solve this type of problem, we need a K-NN algorithm.
- With the help of K-NN, we can easily identify the category or class of a particular dataset

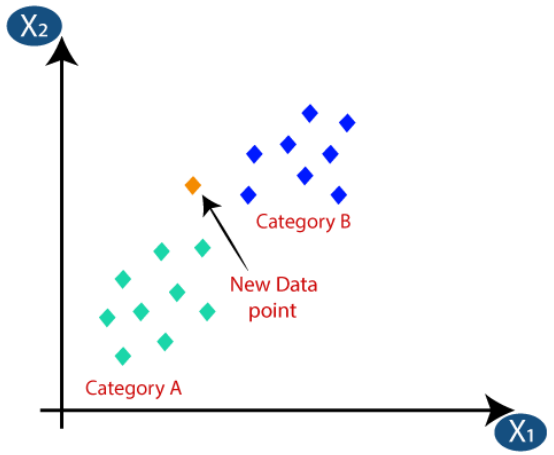


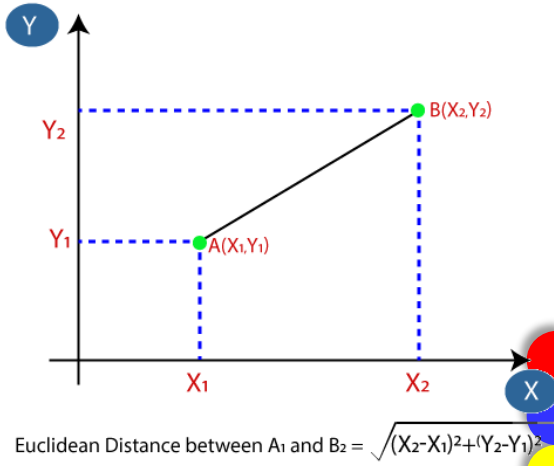
How does K-NN work?

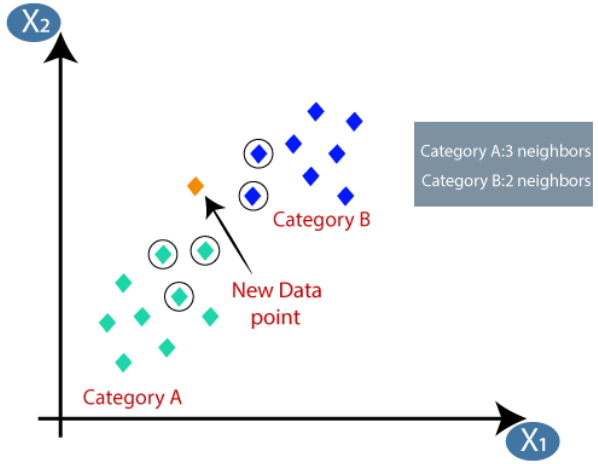
- The K-NN working can be explained on the basis of the below algorithm:
- Step-1: Select the number K of the neighbors
- Step-2: Calculate the Euclidean distance of K number of neighbors, Arrange it in ascending order
- Step-3: Take the K nearest neighbors as per the calculated Euclidean distance.
- Step-4: Among these k neighbors, count the number of the data points in each category.
- Step-5: Assign the new data points to that category for which the number of the neighbor is maximum.
- Step-6: Our model is ready.

K-Nearest Neighbor(KNN)

- choose the number of neighbors, choose the k=5.
- calculate the **Euclidean distance** between the data points.
- Euclidean distance we got three nearest neighbors in category A and two nearest neighbors in category B.
- new data point must belong to category A.







Brightness	Saturation	Class
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

- Here's the new data entry:

Brightness	Saturation	Class
20	35	?

- **Distance #1**
- For the first row, d1:

Brightness	Saturation	Class
40	20	Red

- $d1 = \sqrt{(20 - 40)^2 + (35 - 20)^2}$
  $= \sqrt{400 + 225}$
  $= \sqrt{625}$
  $= 25$

Brightness	Saturation	Class	Distance
40	20	Red	25
50	50	Blue	?
60	90	Blue	?
10	25	Red	?
70	70	Blue	?
60	10	Red	?
25	80	Blue	?

- **Distance #2**
- For the second row, d2:
- $d2 = \sqrt{(20 - 50)^2 + (35 - 50)^2}$
  $= \sqrt{900 + 225}$
  $= \sqrt{1125}$
  $= 33.54$
- **Distance #3**
- For the third row, d3:
- $d2 = \sqrt{(20 - 60)^2 + (35 - 90)^2}$
  $= \sqrt{1600 + 3025}$
  $= \sqrt{4625}$
  $= 68.01$

Brightness	Saturation	Class	Distance
40	20	Red	25
50	50	Blue	33.54
60	90	Blue	68.01
10	25	Red	10
70	70	Blue	61.03
60	10	Red	47.17
25	80	Blue	45

- Let's rearrange the distances in ascending order:

Brightness	Saturation	Class	Distance
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17
70	70	Blue	61.03
60	90	Blue	68.01

- Since we chose 5 as the value of **K**, we'll only consider the first five rows. That is:

- As you can see above, the majority class within the 5 nearest neighbors to the new entry is **Red**. Therefore, we'll classify the new entry as **Red**.

Brightness	Saturation	Class	Distance
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17

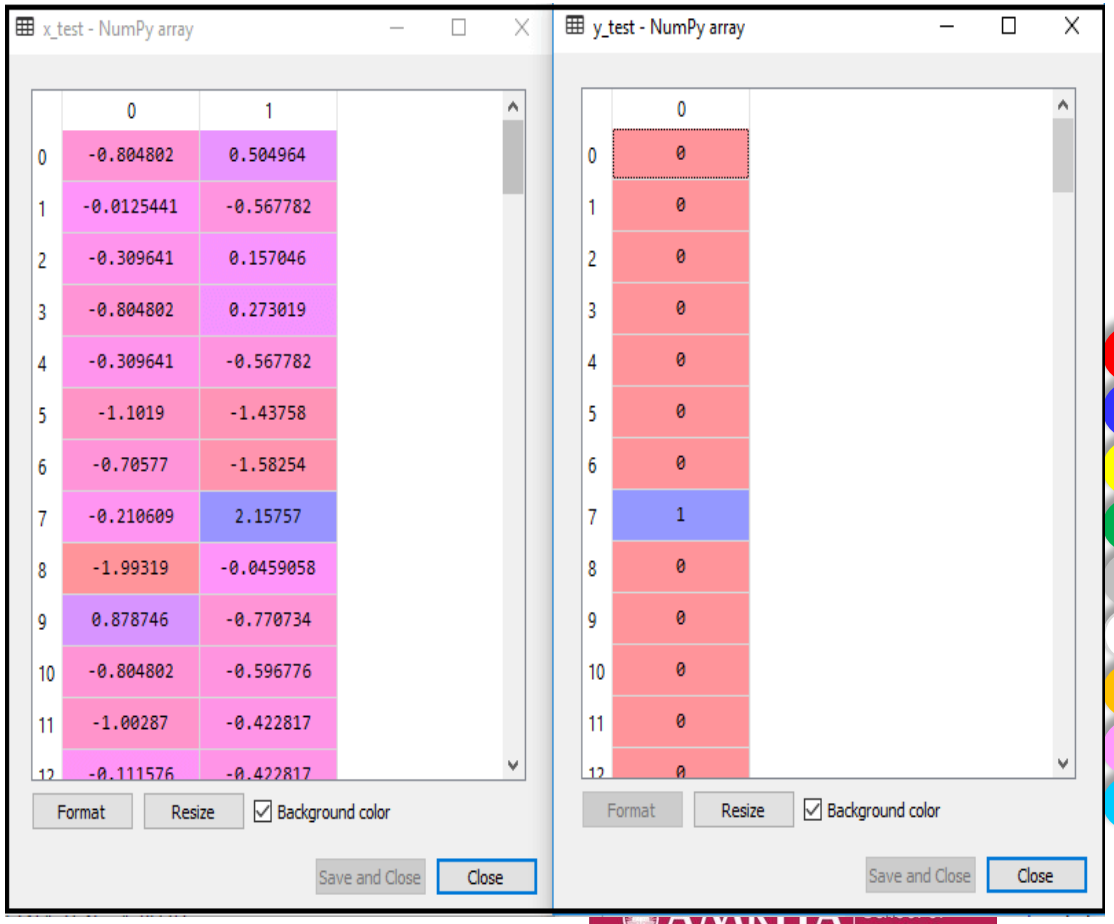
K-Nearest Neighbor(KNN)

- The dataset contains **Estimated Salary** and **Age** -consider for the independent variable and the **Purchased variable-** dependent variable.

User ID	Gender	Age	EstimatedSalary	Purchased
15624510	Male	19	19000	0
15810944	Male	35	20000	0
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	0
15728773	Male	27	58000	0
15598044	Female	27	84000	0
15694829	Female	32	150000	1
15600575	Male	25	33000	0
15727311	Female	35	65000	0
15570769	Female	26	80000	0
15606274	Female	26	52000	0
15746139	Male	20	86000	0
15704987	Male	32	18000	0
15628972	Male	18	82000	0
15697686	Male	29	80000	0
15733883	Male	47	25000	1
15617482	Male	45	26000	1
15704583	Male	46	28000	1
15621083	Female	48	29000	1
15649487	Male	45	22000	1
15736760	Female	47	49000	1

K-Nearest Neighbor(KNN)

```
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
  #importing datasets
data_set= pd.read_csv('user_data.csv')
  #Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values
  # Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)
  #feature Scaling
from sklearn.preprocessing import StandardScaler
st_x= StandardScaler()
x_train= st_x.fit_transform(x_train)
x_test= st_x.transform(x_test)
```



	0	1
0	-0.804802	0.504964
1	-0.0125441	-0.567782
2	-0.309641	0.157046
3	-0.804802	0.273019
4	-0.309641	-0.567782
5	-1.1019	-1.43758
6	-0.70577	-1.58254
7	-0.210609	2.15757
8	-1.99319	-0.0459058
9	0.878746	-0.770734
10	-0.804802	-0.596776
11	-1.00287	-0.422817
12	-0.111576	-0.422817

	0
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	0
9	0
10	0
11	0
12	0

- **Fitting K-NN classifier to the Training data:**
  import the **KNeighborsClassifier** class of **Sklearn Neighbors** library.
- The Parameter of this class will be**n_neighbors:** To define the required neighbors of the algorithm. Usually, it takes 5.
- **metric='minkowski':** This is the default parameter and it decides the distance between the points.
- **p=2:** It is equivalent to the standard Euclidean metric.

- #Fitting K-NN classifier to the training set
- from sklearn.neighbors **import** KNeighbors Classifier
- classifier= KNeighborsClassifier(n_neighbors =5, metric='minkowski', p=2)
- classifier.fit(x_train, y_train)

K-Nearest Neighbor(KNN)

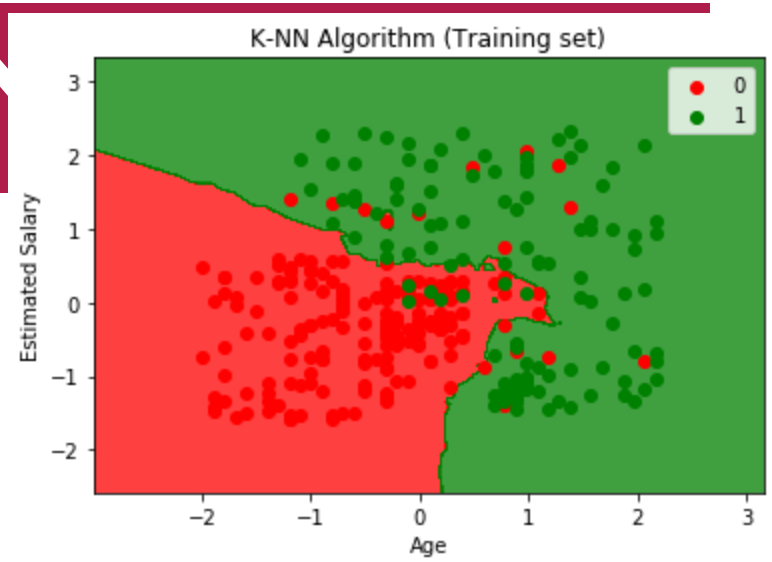
- **Predicting the Test Result:** To predict the test set result, we will create a **y_pred** vector as we did in Logistic Regression. Below is the code for it:
- #Predicting the test set result
- y_pred= classifier.predict(x_test)

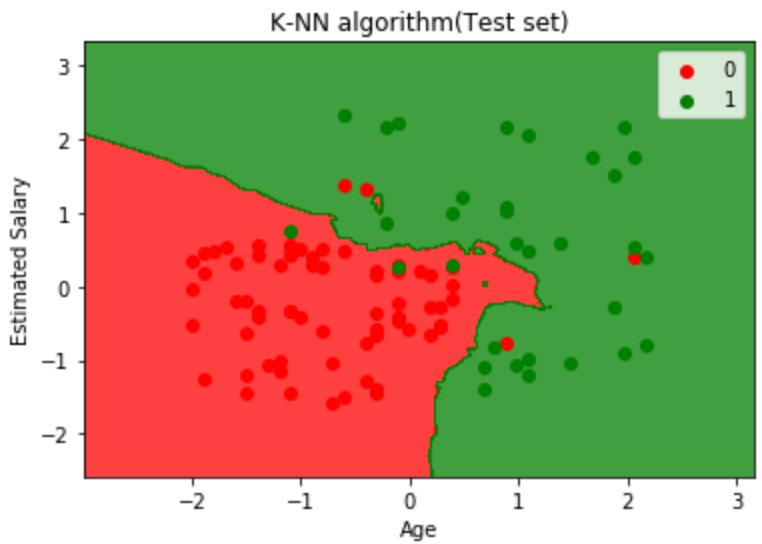
K-Nearest Neighbor(KNN)

- **Creating the Confusion Matrix:**
  Now we will create the Confusion Matrix for our K-NN model to see the accuracy of the classifier. Below is the code for it:
- #Creating the Confusion matrix
- from sklearn.metrics **import** confusion_ matrix
- cm= confusion_matrix(y_test, y_pred)

K-Nearest Neighbor(KNN

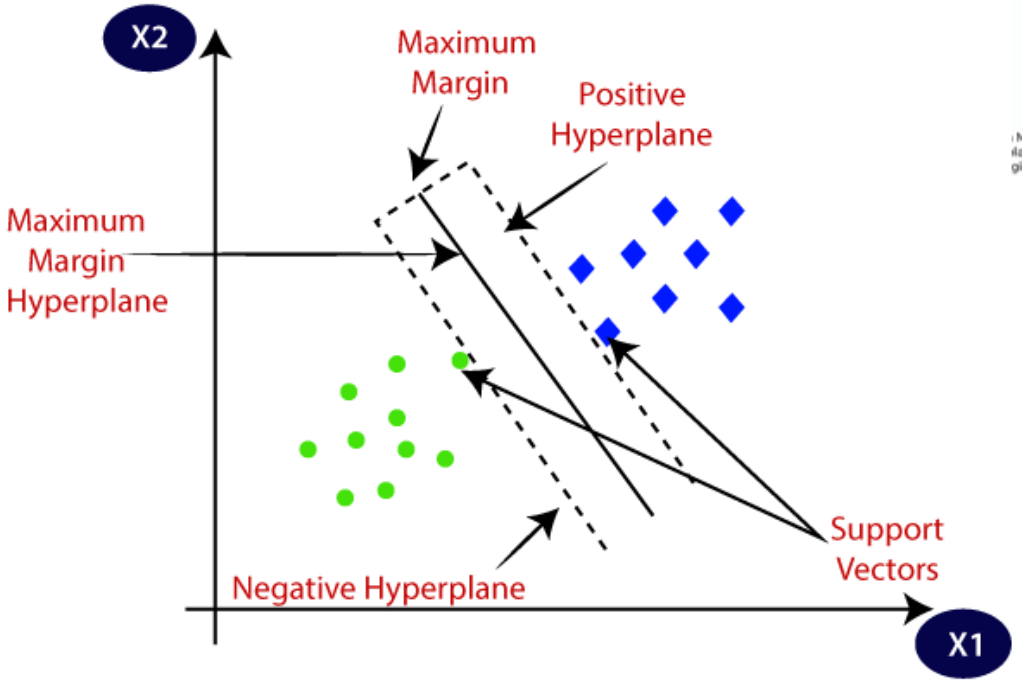
- The green points are for Purchased(1) and Red Points for not Purchased(0) variable.
- The graph is showing an irregular boundary instead of showing any straight line or any curve because it is a K-NN algorithm.
- The graph is showing good result but still, there are some green points in the red region and red points in the green region.





Support Vector Machine

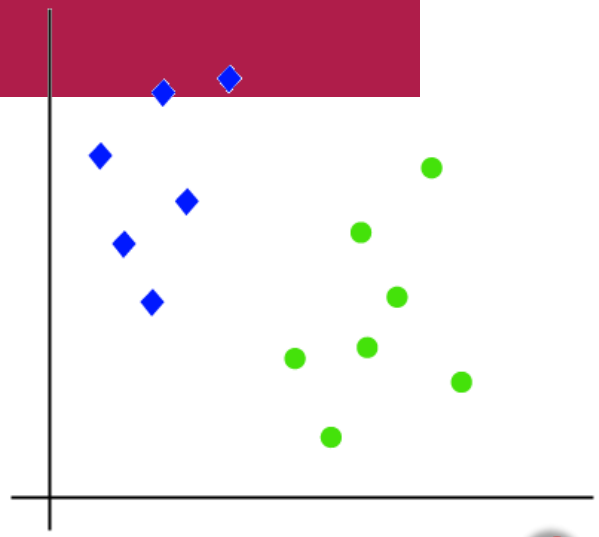
- Support Vector Machine or SVM Supervised Learning algorithms used for Classification, Regression problems.
- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future.
- This best decision boundary is called a hyperplane.
- SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine



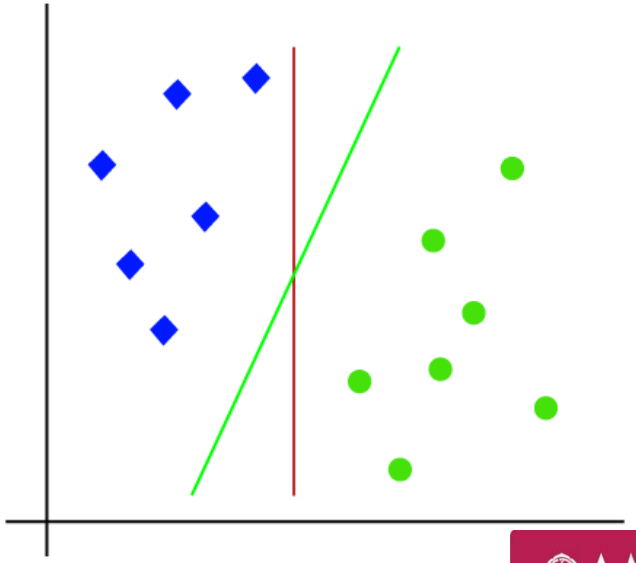
Support Vector Machine

- Types of SVM
- **SVM can be of two types:**
- **Linear SVM:** Linear SVM is used for linearly separable data-dataset can be classified into two classes by using a single straight line and classifier - Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, dataset cannot be classified by using a straight line, classifier called as Non-linear SVM classifier.
- **Hyperplane:** There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.
- The dimensions of the hyperplane depend on the features present in the dataset, if there are 2 features , then hyperplane will be a straight line.
- And if there are 3 features, then hyperplane will be a 2-dimension plane.

Support Vector Machine

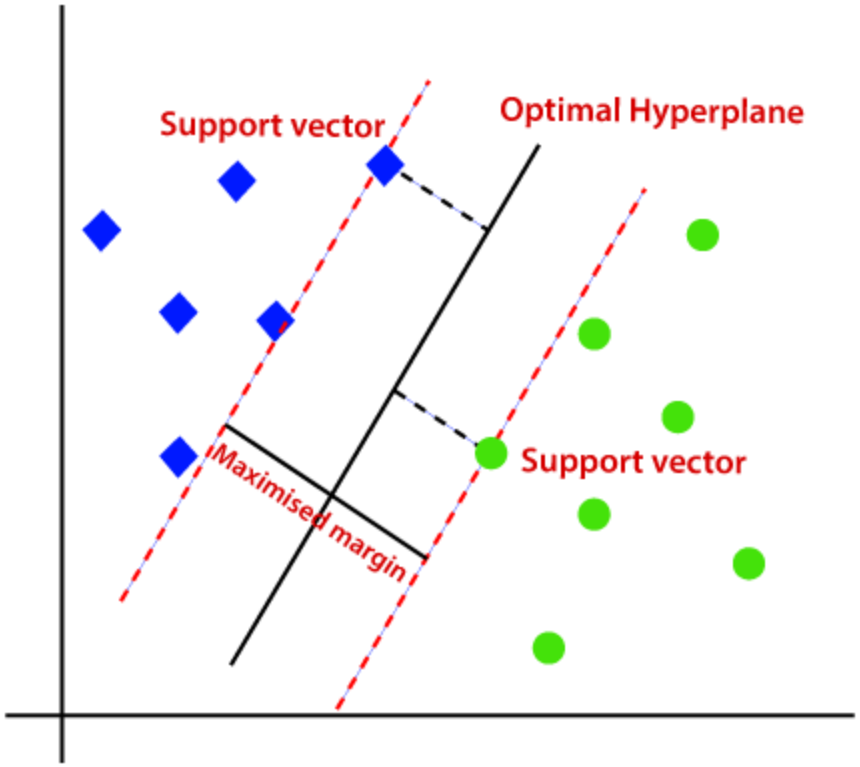


- How does SVM works?
- **Linear SVM:**
- Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x1 and x2.
- We want a classifier that can classify the pair(x1, x2) of coordinates in either green or blue.
- it is 2-d space so by just using a straight line,can easily separate these two classes.
- But there can be multiple lines that can separate these classes.



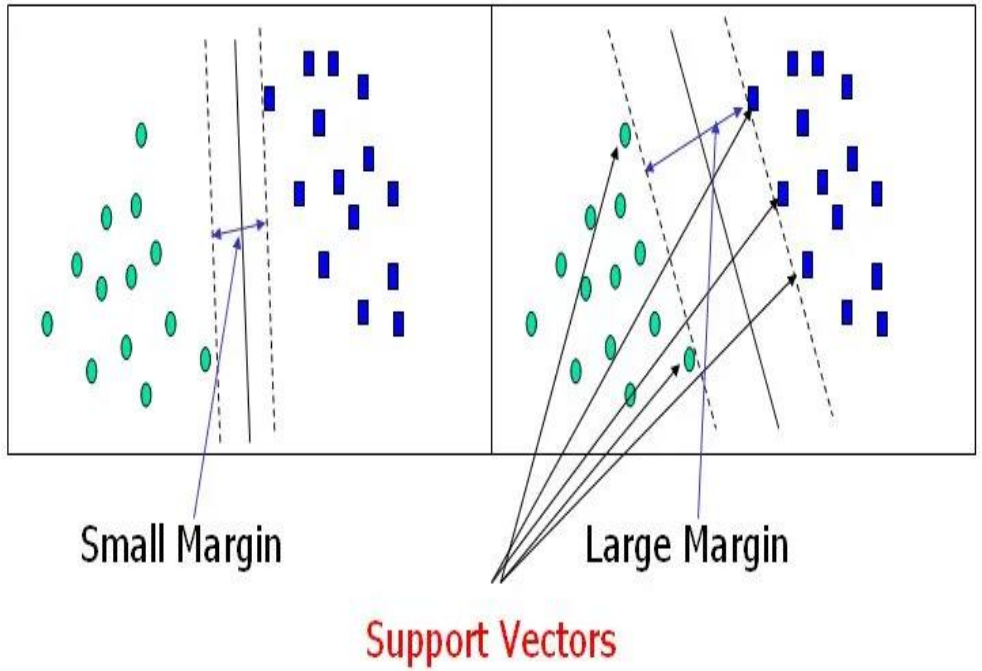
Support Vector Machine

- SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**.
- SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors.
- The distance between the vectors and the hyperplane is called as **margin**.
- Goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



Support Vectors

- Support vectors are data points that are closer to the hyperplane and influence the position and orientation of the hyperplane.
- Using these support vectors, we maximize the margin of the classifier.
- Deleting the support vectors will change the position of the hyperplane.
- These are the points that help us build our SVM.



Large Margin Intuition, Cost Function and Gradient Updates

- In SVM, we take the output of the linear function and if that output is greater than 1, we identify it with one class and if the output is -1, we identify is with another class.
- Since the threshold values are changed to 1 and -1 in SVM, we obtain this reinforcement range of values([-1,1]) which acts as margin.
- In the SVM algorithm, we are looking to maximize the margin between the data points and the hyperplane. The loss function that helps maximize the margin is hinge loss.
- The cost is 0 if the predicted value and the actual value are of the same sign. If they are not, we then calculate the loss value. We also add a regularization parameter the cost function.

$$c(x, y, f(x)) = \begin{cases} 0, & \text{if } y * f(x) \geq 1 \\ 1 - y * f(x), & \text{else} \end{cases}$$

- he objective of the regularization parameter is to balance the margin maximization and loss. After adding the regularization parameter, the cost functions looks as below.

$$min_w \lambda \parallel w \parallel^2 + \sum_{i=1}^{n} (1 - y_i \langle x_i, w \rangle)_+$$

- Now that we have the loss function, we take partial derivatives with respect to the weights to find the gradients. Using the gradients, we can update our weights.
-

$$\frac{\delta}{\delta w_k} \lambda \parallel w \parallel^2 = 2\lambda w_k$$

$$\frac{\delta}{\delta w_k} \left(1 - y_i \langle x_i, w \rangle\right)_+ = \begin{cases} 0, & \text{if } y_i \langle x_i, w \rangle \geq 1 \\ -y_i x_{ik}, & \text{else} \end{cases}$$

- When there is no misclassification, i.e our model correctly predicts the class of our data point, we only have to update the gradient from the regularization parameter.

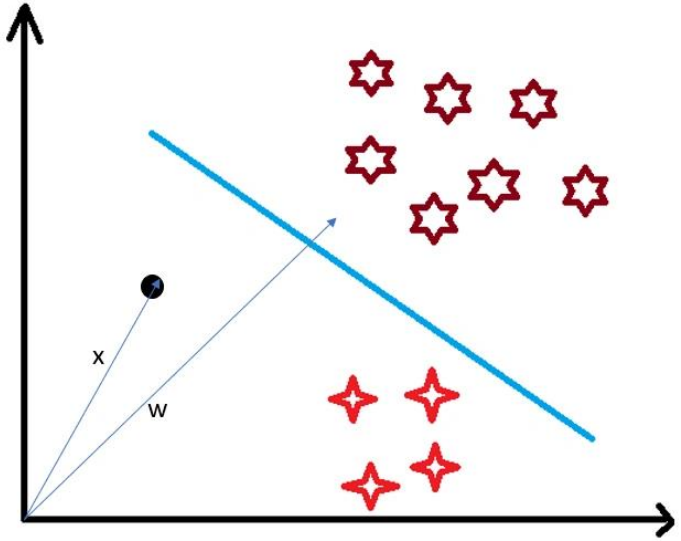
$$w = w - \alpha \cdot (2\lambda w)$$

- When there is a misclassification, i.e our model make a mistake on the prediction of the class of our data point, we include the loss along with the regularization parameter to perform gradient update.

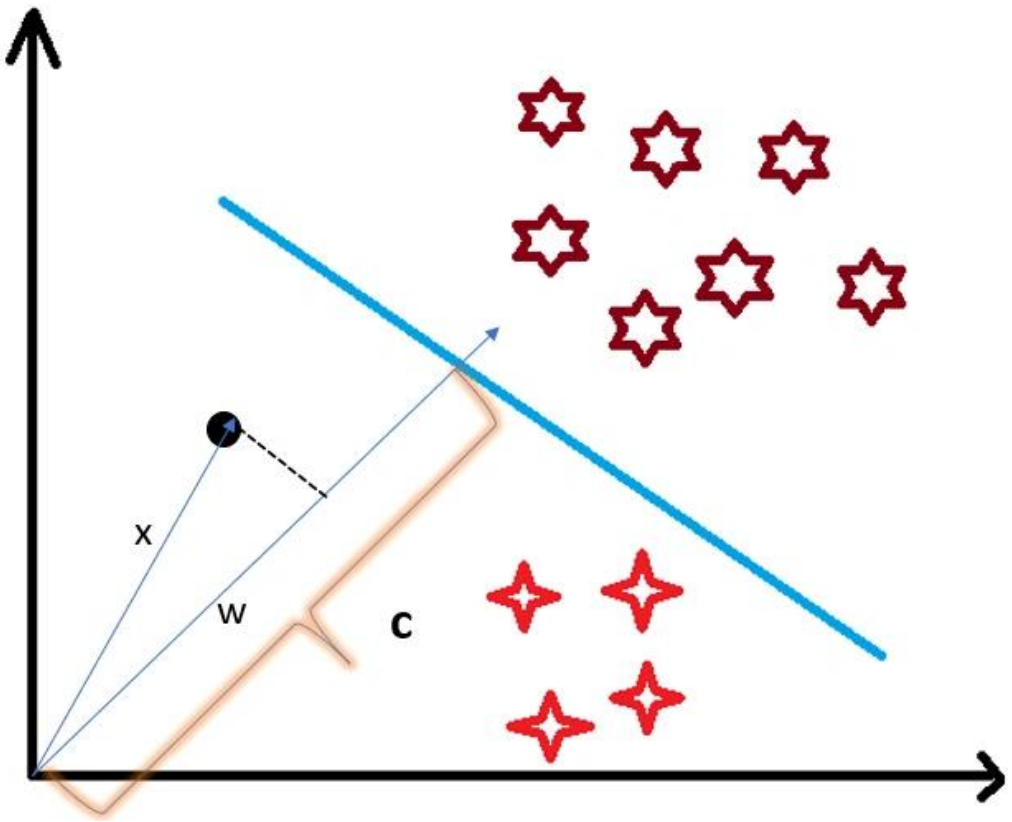
$$w = w + \alpha \cdot (y_i \cdot x_i - 2\lambda w)$$

- The dot product can be defined as the projection of one vector along with another, multiply by the product of another vector.
- After finding the magnitude we simply multiply it with the cosine angle between both the vectors. Mathematically it can be written as:
- **A . B = |A| cosθ * |B|**
- Where |A| cosθ is the projection of A on B
- And |B| is the magnitude of vector B

- Use of Dot Product in SVM
- Consider a random point X and we want to know whether it lies on the right side of the plane or the left side of the plane (positive or negative).

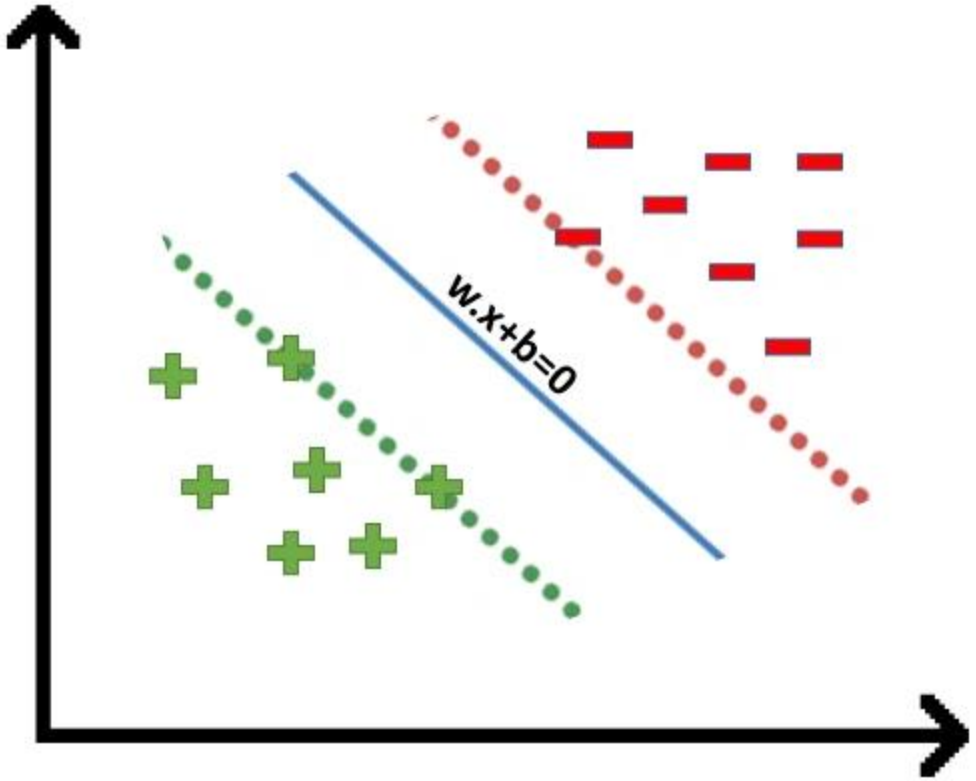


- To find this first we assume this point is a vector (X) and then we make a vector (w) which is perpendicular to the hyperplane.
- Let's say the distance of vector w from origin to decision boundary is 'c'. Now we take the projection of X vector on w.
- We already know that projection of any vector or another vector is called dot-product.
- Hence, we take the dot product of x and w vectors. If the dot product is greater than 'c' then we can say that the point lies on the right side.
- If the dot product is less than 'c' then the point is on the left side and if the dot product is equal to 'c' then the point lies on the decision boundary.

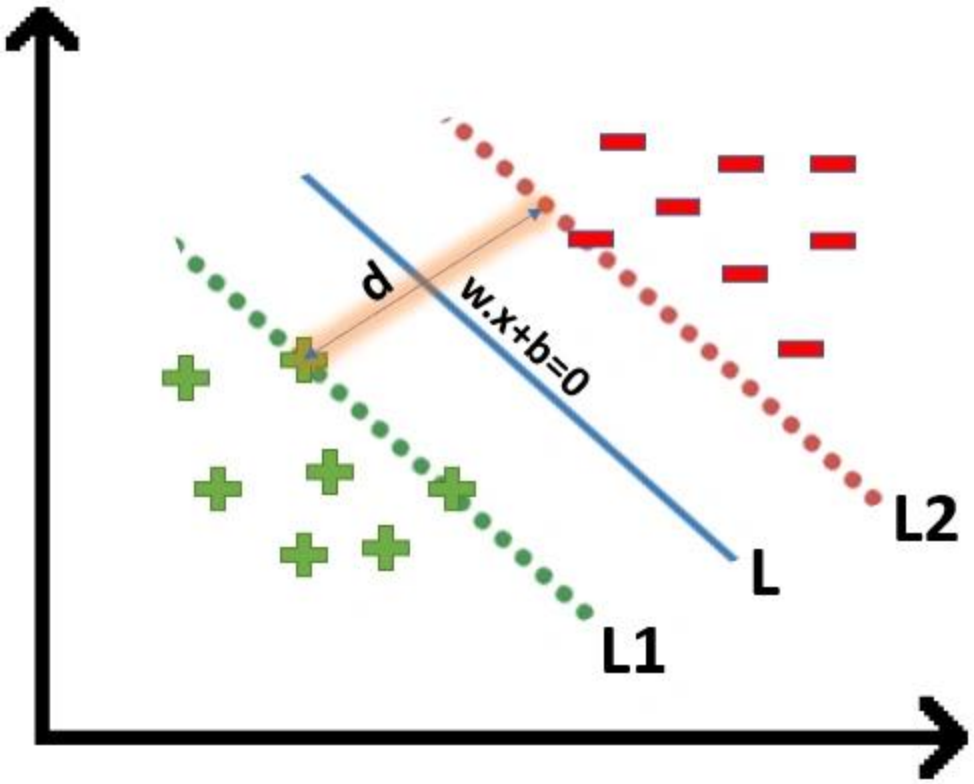


Margin in Support Vector Machine

- We all know the equation of a hyperplane is w.x+b=0 where w is a vector normal to hyperplane and b is an offset.
- If the value of w.x+b>0 then we can say it is a positive point otherwise it is a negative point. Now we need (w,b) such that the margin has a maximum distance. Let's say this distance is 'd'.

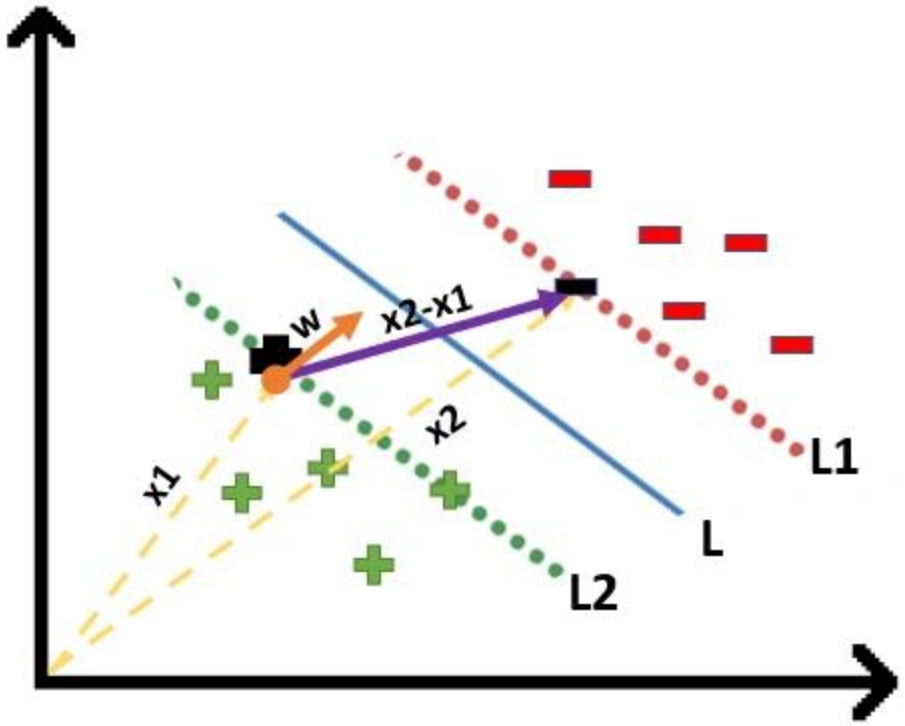


- To calculate ‘d’ we need the equation of L1 and L2. For this, we will take few assumptions that the equation of L1 is **w.x+b=1** and for L2 it is **w.x+b=-1**.
- We know that the aim of SVM is to maximize this margin that means distance (d). But there are few constraints for this distance (d).



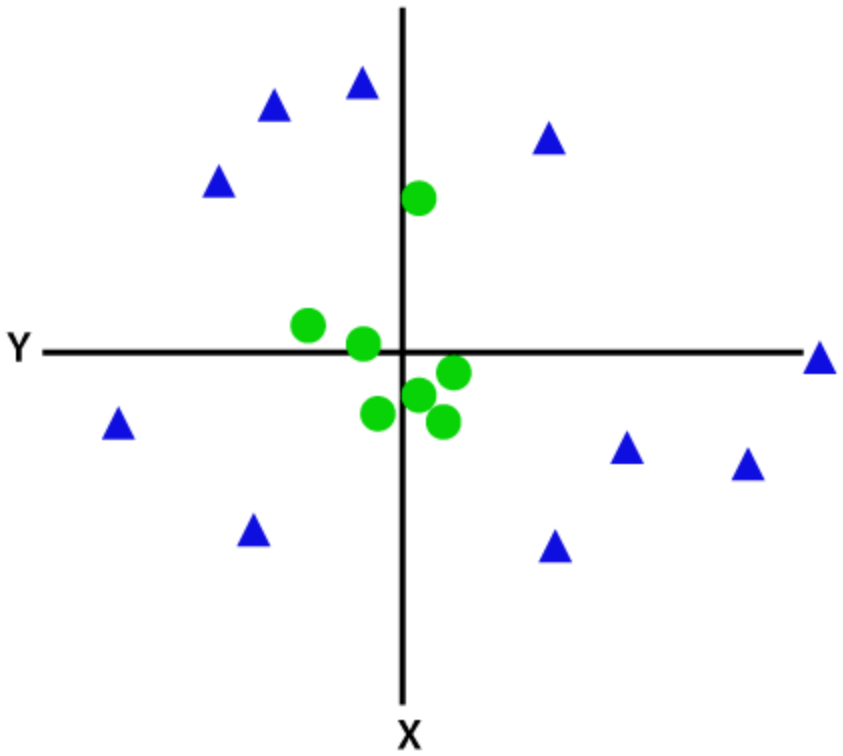
Constraints

- That constraint is that "We'll calculate the distance (d) in such a way that no positive or negative point can cross the margin line".
- We will take 2 support vectors, 1 from the negative class and 2nd from the positive class. The distance between these two vectors x1 and x2 will be *(x2-x1) vector*.
- What we need is, the shortest distance between these two points which can be found using a trick we used in the dot product. We take a vector 'w' perpendicular to the hyperplane and then find the projection of (x2-x1) vector on 'w'.



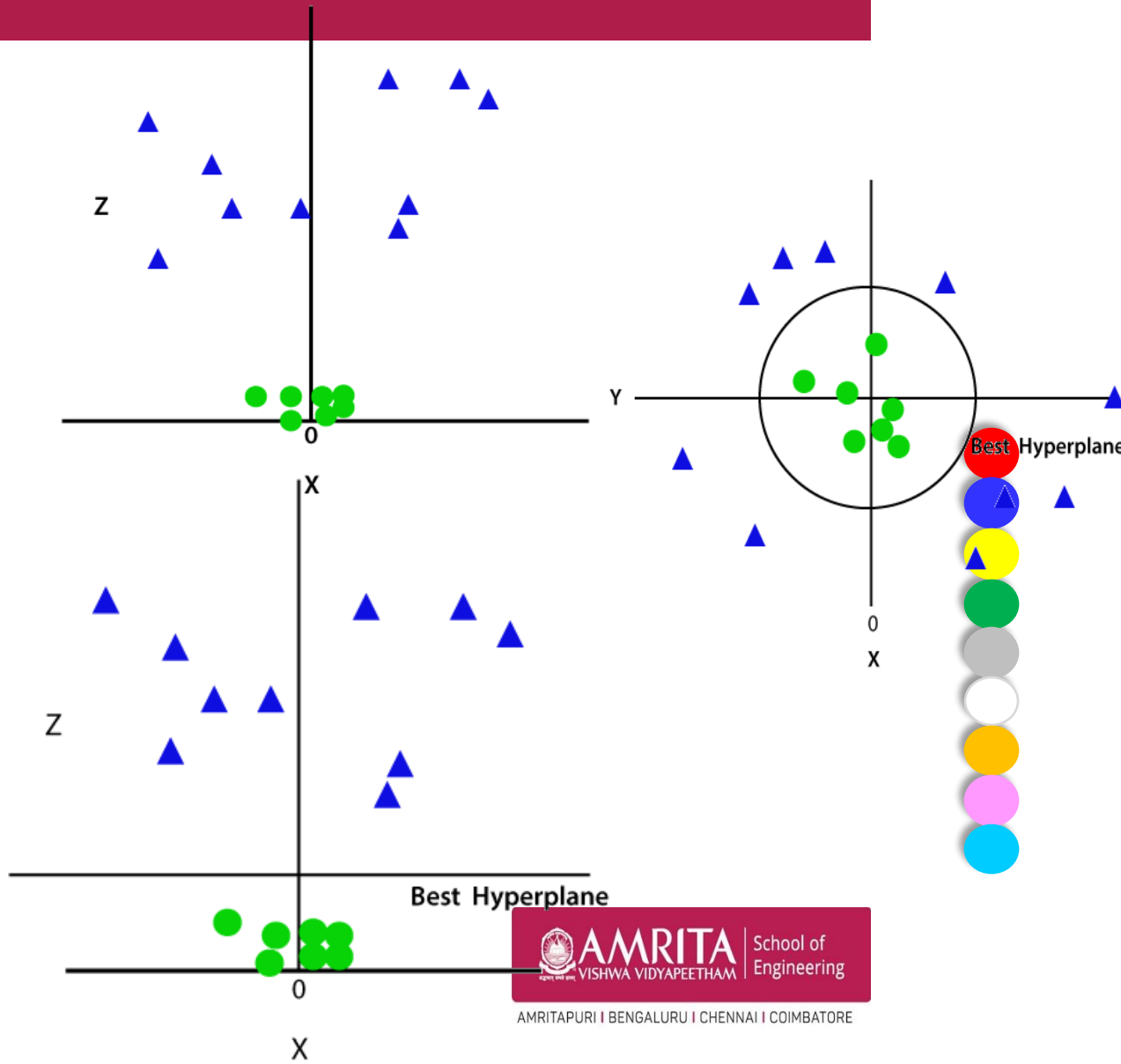
Support Vector Machine

- **Non-Linear SVM:**
- If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line.
- So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as: z=x2 +y2



Support Vector Machine

- By adding the third dimension, the sample space will become like in figure.
- SVM will divide the datasets into classes
- Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis.



Support Vector Machine

- **Data Pre-processing step**
- # importing libraries
- **import** numpy as nm
- **import** matplotlib.pyplot as mtp
- **import** pandas as pd
- #importing datasets
- data_set= pd.read_csv('user_data.csv')
- #Extracting Independent and dependent Variable
- x= data_set.iloc[:, [2,3]].values
- y= data_set.iloc[:, 4].values
- # Splitting the dataset into training and test set.
- from sklearn.model_selection **import** train_test_split
- x_train, x_test, y_train, y_test = train_test_split(x, y, test_size= 0.25, random_state=0)
- #feature Scaling
- from sklearn.preprocessing **import** StandardScaler
- st_x= StandardScaler()
- x_train= st_x.fit_transform(x_train)
- x_test= st_x.transform(x_test)

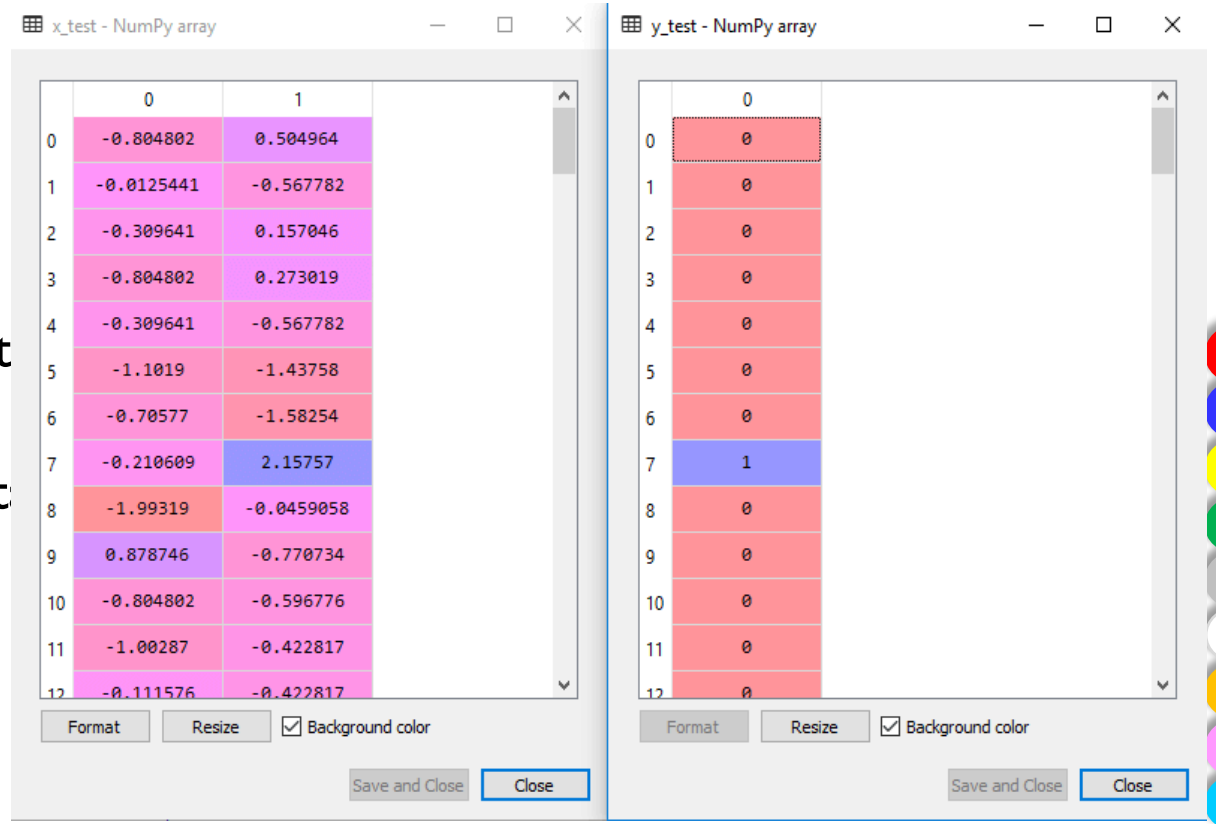
data_set - DataFrame

Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0
14	15628972	Male	18	82000	0

Format | Resize | Background color | Column min/max | Save and Close | Close

Support Vector Machine

- The scaled output for the test set will be:

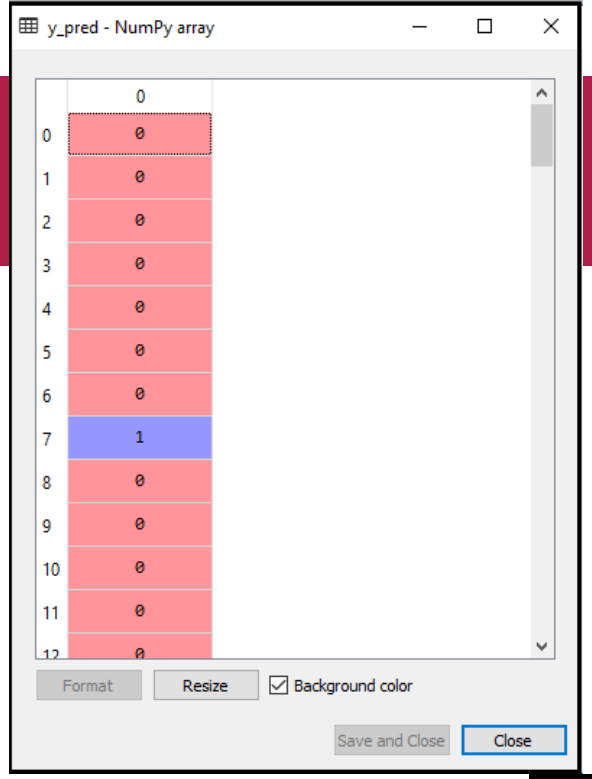


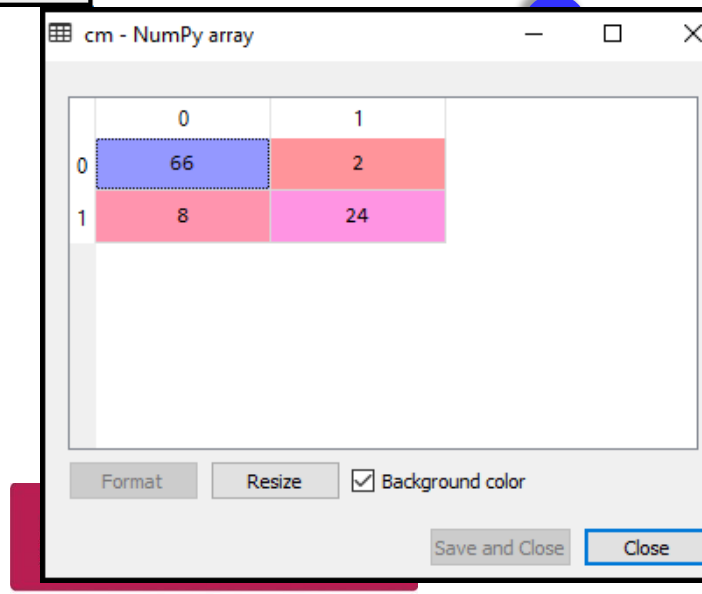
	0	1
0	-0.804802	0.504964
1	-0.0125441	-0.567782
2	-0.309641	0.157046
3	-0.804802	0.273019
4	-0.309641	-0.567782
5	-1.1019	-1.43758
6	-0.70577	-1.58254
7	-0.210609	2.15757
8	-1.99319	-0.0459058
9	0.878746	-0.770734
10	-0.804802	-0.596776
11	-1.00287	-0.422817
12	-0.111576	-0.422817

	0
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	0
9	0
10	0
11	0
12	0

- **Fitting the SVM classifier to the training set:**
- from sklearn.svm **import** SVC # "Support ector classifier"
- classifier = SVC(kernel='linear', random_st e=0)
- classifier.fit(x_train, y_train)
- **kernel='linear'-** linearly separable data.

- **Predicting the test set result:**
- #Predicting the test set result
- y_pred= classifier.predict(x_test)
- **Creating the confusion matrix:**
- #Creating the Confusion matrix
- from sklearn.metrics **import** confusion_matrix
- cm= confusion_matrix(y_test, y_pred)
-





Naïve Bayes Classifier

- Naïve Bayes algorithm is a supervised learning algorithm, based on **Bayes theorem** and used for solving classification.
- **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object**.
- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles**.

- Why is it called Naïve Bayes?
- **Naïve**: It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features.
- **Bayes**: It is called Bayes because it depends on the principle of Bayes' Theorem

Naïve Bayes

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- Imagine you want to classify an email as spam (A="Spam") or not spam based on whether it contains the word "free" (B="Word 'free' appears").
- **Prior Probability P(A)**
  Suppose 40% of all emails are spam, so P(Spam)=0.4
- **Likelihood P(B|A):**
  Among spam emails, 70% contain the word "free." Hence, P('free'|Spam)=0.7
- **Marginal Probability P(B):**
  Out of all emails (spam and non-spam), 30% contain the word "free." So P('free')=0.3
- **Posterior Probability P(A|B):**
  You want to calculate P(Spam|'free'): the probability that an email is spam given it contains the word "free.

How Naive Bayes Classifier Works?

- **First Approach (In case of a single feature)**
- Naive Bayes classifier calculates the probability of an event in the following steps:
- **Step 1**: Calculate the prior probability for given class labels
- **Step 2**: Find Likelihood probability with each attribute for each class
- **Step 3**: Put these value in Bayes Formula and calculate posterior probability.
- **Step 4**: See which class has a higher probability, given the input belongs to the higher probability class.

- Now suppose you want to calculate the probability of playing when the weather is overcast.
- **Probability of playing:**
- *P(Yes | Overcast) = P(Overcast | Yes)* P(Yes) / P (Overcast) .....(1)
- Calculate Prior Probabilities:
- P(Overcast) = 4/14 = 0.29
- P(Yes)= 9/14 = 0.64
- Calculate Posterior Probabilities:
- P(Overcast |Yes) = 4/9 = 0.44
- Put Prior and Posterior probabilities in equation (1)
- P (Yes | Overcast) = 0.44 * 0.64 / 0.29 = 0.98(Higher)

- **Probability of not playing:**
- *P(No | Overcast) = P(Overcast | No)* P(No) / P (Overcast) .....(2)
- Calculate Prior Probabilities:
- P(Overcast) = 4/14 = 0.29
- P(No)= 5/14 = 0.36
- Calculate Posterior Probabilities:
- P(Overcast |No) = 0/9 = 0
- Put Prior and Posterior probabilities in equation (2)
- P (No | Overcast) = 0 * 0.36 / 0.29 = 0

Second Approach (In case of multiple features)

HOW NAIVE BAYES CLASSIFIER WORKS?

Whether	Temperature	Play
Sunny	Hot	No
Sunny	Hot	No
Overcast	Hot	Yes
Rainy	Mild	Yes
Rainy	Cool	Yes
Rainy	Cool	No
Overcast	Cool	Yes
Sunny	Mild	No
Sunny	Cool	Yes
Rainy	Mild	Yes
Sunny	Mild	Yes
Overcast	Mild	Yes
Overcast	Hot	Yes
Rainy	Mild	No

01 CALCULATE PRIOR PROBABILITY FOR GIVEN CLASS LABELS

02 CALCULATE CONDITIONAL PROBABILITY WITH EACH ATTRIBUTE FOR EACH CLASS

03 MULTIPLY SAME CLASS CONDITIONAL PROBABILITY.

04 MULTIPLY PRIOR PROBABILITY WITH STEP 3 PROBABILITY.

05 SEE WHICH CLASS HAS HIGHER PROBABILITY, HIGHER PROBABILITY CLASS BELONGS TO GIVEN INPUT SET STEP.

the probability of playing (P(Play = Yes)) based on the weather (P(Weather = Overcast)) and temperature (P(Temp = Mild))

- **Probability of playing:**
- *P(Play= Yes | Weather=Overcast, Temp=Mild) = P(Weather=Overcast, Temp=Mild | Play= Yes)* P(Play=Yes) .....(1)
- *P(Weather=Overcast, Temp=Mild | Play= Yes)= P(Overcast |Yes)* P(Mild |Yes) ……..(2)
- Calculate Prior Probabilities: P(Yes)= 9/14 = 0.64
- Calculate Posterior Probabilities: P(Overcast |Yes) = 4/9 = 0.44 P(Mild |Yes) = 4/9 = 0.44
- Put Posterior probabilities in equation (2)
- P(Weather=Overcast, Temp=Mild | Play= Yes) = 0.44 * 0.44 = 0.1936(Higher)
- Put Prior and Posterior probabilities in equation (1)
- P(Play= Yes | Weather=Overcast, Temp=Mild) = 0.1936*0.64 = 0.124

- **Probability of not playing:**
- *P(Play= No | Weather=Overcast, Temp=Mild) = P(Weather=Overcast, Temp=Mild | Play= No)* P(Play=No) .....(3)
- *P(Weather=Overcast, Temp=Mild | Play= No)= P(Weather=Overcast |Play=No)* P(Temp=Mild | Play=No) ……..(4)
- Calculate Prior Probabilities: P(No)= 5/14 = 0.36
- Calculate Posterior Probabilities: P(Weather=Overcast |Play=No) = 0/9 = 0 P(Temp=Mild | Play=No)=2/5=0.4
- Put posterior probabilities in equation (4) P(Weather=Overcast, Temp=Mild | Play= No) = 0 * 0.4= 0
- Put prior and posterior probabilities in equation (3) P(Play= No | Weather=Overcast, Temp=Mild) = 0*0.36=0
- *The probability of a 'Yes' class is higher.*

Naïve Bayes

- Working of Naïve Bayes' Classifier:
- Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**".
- So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions.
- Convert the given dataset into frequency tables.
- Generate Likelihood table by finding the probabilities of given features.
- Now, use Bayes theorem to calculate the posterior probability

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Naïve Bayes

- **Frequency table for the Weather Conditions:Likelihood table weather condition:**
- **Applying Bayes'theorem:**
- **P(Yes|Sunny)= P(Sunny|Yes)*P(Yes)/P(Sunny)**
- P(Sunny|Yes)= 3/10= 0.3
- P(Sunny)= 0.35
- P(Yes)=0.71
- So P(Yes|Sunny) = 0.3*0.71/0.35= **0.60**
- **P(No|Sunny)= P(Sunny|No)*P(No)/P(Sunny)**
- P(Sunny|NO)= 2/4=0.5
- P(No)= 0.29
- P(Sunny)= 0.35
- So P(No|Sunny)= 0.5*0.29/0.35 = **0.41**
- So as we can see from the above calculation that **P(Yes|Sunny)>P(No|Sunny)**
- **Hence on a Sunny day, Player can play the game.**

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	5

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

Naïve Bayes

- Types of Naïve Bayes Model:
- **Gaussian**: The Gaussian model assumes that features follow a normal distribution.
- if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- student will pass or fail an exam based on their study hours
- **Multinomial**: The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed.
- It is primarily used for document classification problems-document belongs to which category such as Sports, Politics, education, etc.
  Imagine you are building a spam email classifier.
- You use the frequency of words like "win," "free," and "offer" in emails.
- The classifier calculates the probability of each word appearing in spam vs. non-spam emails.
- **Bernoulli**: The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables.
- Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.
- same spam email classification task: Instead of word frequencies, the model checks whether a word like "win" or "free" is present (1) or absent (0) in an email.

Naïve Bayes

- Data Pre-processing step:
- import numpy as nm
- **import** matplotlib.pyplot as mtp
- **import** pandas as pd
- # Importing the dataset
- dataset = pd.read_csv('user_data.csv')
- x = dataset.iloc[:, [2, 3]].values
- y = dataset.iloc[:, 4].values
- # Splitting the dataset into the Training set and Test set
- **from** sklearn.model_selection **import** train_test_split
- x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.25, random_state = 0)

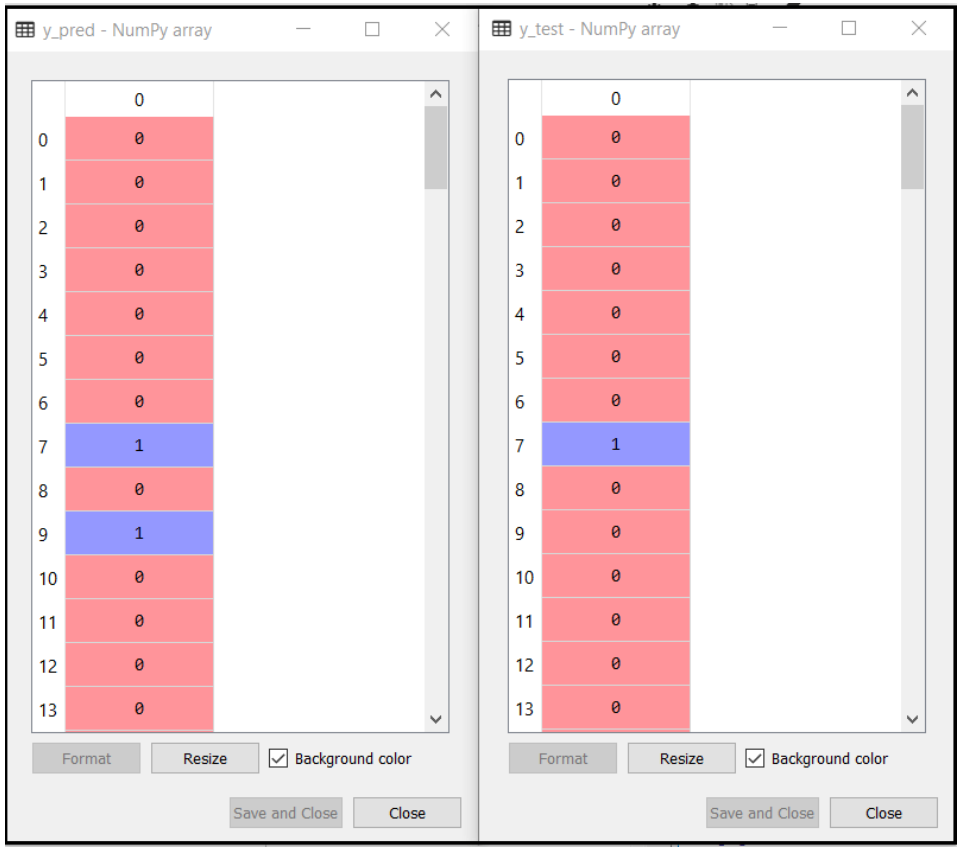
- # Feature Scaling
- **from** sklearn.preprocessing **import** StandardScaler
- sc = StandardScaler()
- x_train = sc.fit_transform(x_train)
- x_test = sc.transform(x_test)

dataset - DataFrame

Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0
14	15628972	Male	18	82000	0
15	15697686	Male	29	80000	0
16	15733883	Male	47	25000	1
17	15617482	Male	45	26000	1
18	15704583	Male	46	28000	1
19	15621083	Female	48	29000	1

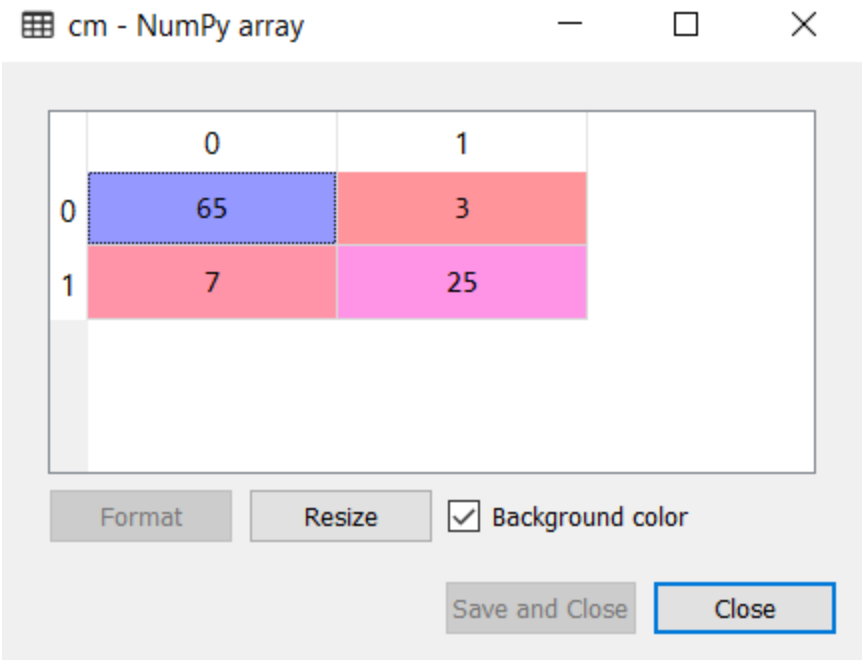
Naïve Bayes

- # Fitting Naive Bayes to the Training set
- **from** sklearn.naive_bayes **import** Gaussian NB
- classifier = GaussianNB()
- classifier.fit(x_train, y_train)
- Prediction of the test set result:
- # Predicting the Test set results
- y_pred = classifier.predict(x_test)



Naïve Bayes

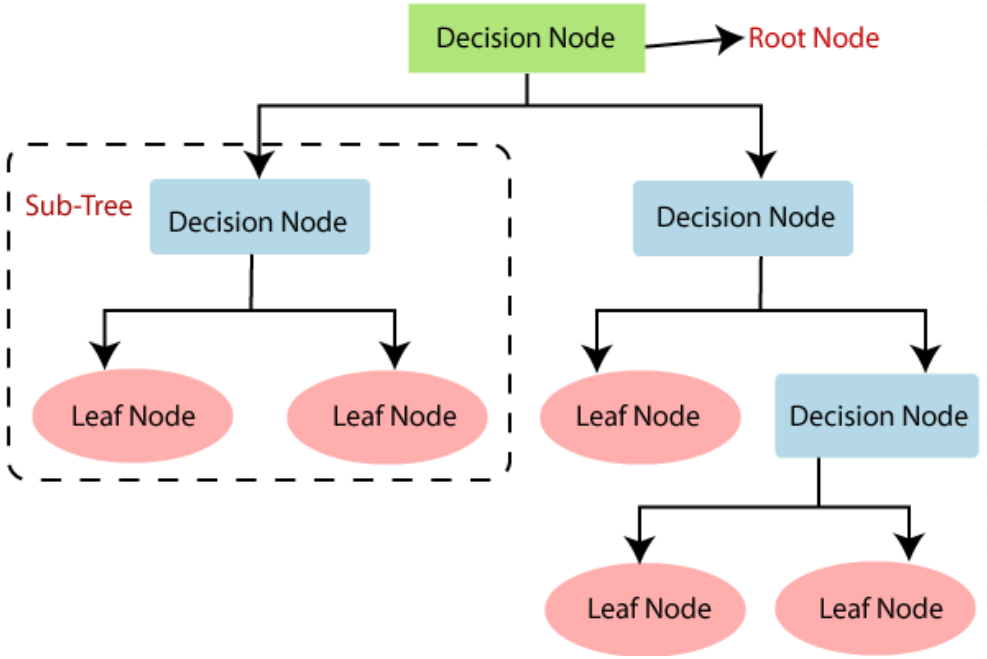
- # Making the Confusion Matrix
- **from** sklearn.metrics **import** confusion_matrix
- cm = confusion_matrix(y_test, y_pred)



Decision Tree Classification

- It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**
- two nodes-**Decision Node** and **Leaf Node.**
- Decision nodes used to make any decision and have multiple branches.
- Leaf nodes are the output of those decisions and do not contain any further branches.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***

- In order to build a tree, use the **CART algorithm,** which stands for **Classification and Regression Tree algorithm.**

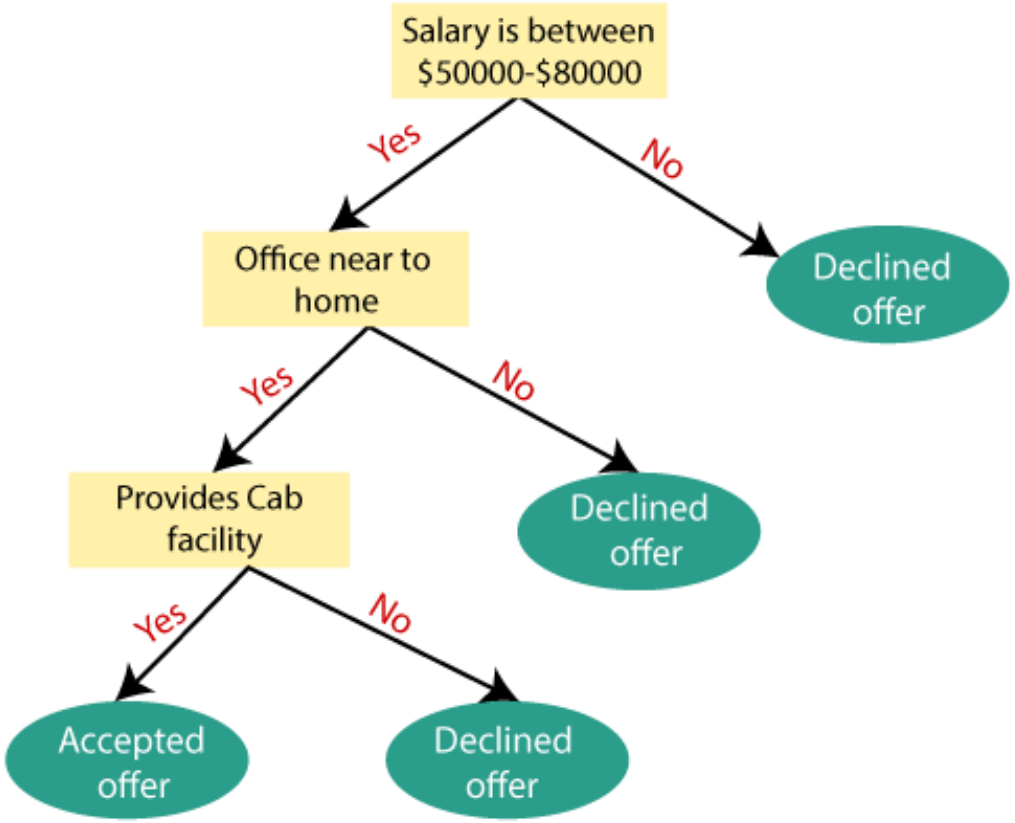


Decision Tree

- Why use Decision Trees?
- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

- Decision Tree Terminologies
- Root Node: Root node is from where the decision tree starts. It represents the entire dataset
- Leaf Node: Leaf nodes are the final output node.
- Splitting: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- Branch/Sub Tree: A tree formed by splitting the tree.
- Pruning: Pruning is the process of removing the unwanted branches from the tree.
- Parent/Child node: The root node of the tree is called the parent node, and other nodes are called the child nodes.

- **How does the Decision Tree algorithm Work?**
- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.



- Attribute Selection Measures
- how to select the best attribute for the root node and for sub-nodes.
- **Information Gain**
- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)

- **Entropy:** Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:
- Entropy(s)= -P(yes)log2 P(yes)- P(no) log2 P(no)
- **Where,**
- **S= Total number of samples**
- **P(yes)= probability of yes**
- **P(no)= probability of no**

- Gini Index:
- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:
- Gini(X)=1−∑pi2 (pi sqaure)
- p i is the probability of each class $ii.nn$ is the total number of classes.

- For the set X = {a,a,a,b,b,b,b,b}
- Total instances: 8
- Instances of b: 5
- Instances of a: 3
- **Step1 Calculate Probabilities**
- p(a)=3/8=0.375,p(b)=5/8=0.625
- **Step 2: Compute Gini Index**
- Gini(X)=1−[p(a)2+p(b)2]

- Gini(X)=1−[(0.375)2+(0.625)2]
- Gini(X)=0.46875

ID3

- ID3 stands for Iterative Dichotomiser 3 and is named such because the algorithm iteratively (repeatedly) dichotomizes(divides) features into two or more groups at each step.

- Most generally ID3 is only used for classification problems with nominal features only.

- ID | Fever | Cough | Breathing issues | Infected |

ID	Fever	Cough	Breathing issues	Infected
1	NO	NO	NO	NO
2	YES	YES	YES	YES
3	YES	YES	NO	NO
4	YES	NO	YES	YES
5	YES	YES	YES	YES
6	NO	YES	NO	NO
7	YES	NO	YES	YES
8	YES	NO	YES	YES
9	NO	YES	YES	YES
10	YES	YES	NO	YES
11	NO	YES	NO	NO
12	NO	YES	YES	YES
13	NO	YES	YES	NO
14	YES	YES	NO	NO

- How does ID3 select the best feature?' is that ID3 uses **Information Gain** or just **Gain** to find the best feature.

- Information Gain calculates the reduction in the entropy and measures how well a given feature separates or classifies the target classes.
- In the case of binary classification (where the target column has only two types of classes) entropy is **0** if all values in the target column are homogenous(similar) and will be **1** if the target column has equal number values for both the classes.

- **$Entropy(S) = - \sum p_i * \log_2(p_i)$; i = 1 to n**
- **S- dataset**
- where,
  n is the total number of classes in the target column (in our case n = 2 i.e YES and NO)
- **p_i** is the **probability of class 'i'** or the ratio of "*number of rows with class i in the target column*" to the "*total number of rows*" in the dataset.
- Information Gain for a feature column **A** is calculated as:
- **$IG(S,A) = Entropy(S) - \sum((|S_v| / |S|) * Entropy(S_v))$**
- where **S_v** is the set of rows in **S** for which the feature column **A** has value **v**, **$|S_v|$** is the number of rows in **S_v** and likewise **|S|** is the number of rows in **S.**

Implementation on our Dataset

- From the total of 14 rows in our dataset **S**, there are **8** rows with the target value **YES** and **6** rows with the target value **NO**. The entropy of **S** is calculated as:
- **Entropy(S) = — (8/14) * $\log_2$(8/14) — (6/14) * $\log_2$(6/14) = 0.99**
- We now calculate the Information Gain for each feature:

- **Fever | Cough | Breathing issues | Infected |**

```
+-------+-------+-----------------+----------+
|YES |YES |YES |YES |
+-------+-------+-----------------+----------+
|YES |YES | NO | NO |
+-------+-------+-----------------+----------+
|YES | NO |YES |YES |
+-------+-------+-----------------+----------+
|YES |YES |YES |YES |
+-------+-------+-----------------+----------+
|YES | NO |YES |YES |
+-------+-------+-----------------+----------+
|YES | NO |YES |YES |
+-------+-------+-----------------+----------+
|YES |YES | NO |YES |
+-------+-------+-----------------+----------+
|YES |YES | NO | NO |
+-------+-------+-----------------+----------+
```

- in the **8** rows with **YES for** Fever, there are **6** rows having target value **YES** and **2** rows having target value **NO.**

- -------+-------+----------------+----------+
  | Fever | Cough | Breathing issues | Infected |
  +-------+-------+----------------+----------+
  | NO | NO | NO | NO |
  +-------+-------+----------------+----------+
  | NO |YES | NO | NO |
  +-------+-------+----------------+----------+
  | NO |YES |YES |YES |
  +-------+-------+----------------+----------+
  | NO |YES | NO | NO |
  +-------+-------+----------------+----------+
  | NO |YES |YES |YES |
  +-------+-------+----------------+----------+
  | NO |YES |YES | NO |
  +-------+-------+----------------+----------+

- in the **6** rows with **NO**, there are **2** rows having target value **YES** and **4** rows having target value **NO.**

calculation of Information Gain for Fever.

total rows

$|S| = 14$

For v = YES, $|S_v| = 8$

$\text{Entropy}(S_v) = -(6/8) * \log_2(6/8) - (2/8) * \log_2(2/8) = 0.81$

For v = NO, $|S_v| = 6$

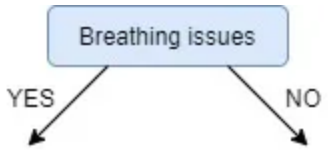
$\text{Entropy}(S_v) = -(2/6) * \log_2(2/6) - (4/6) * \log_2(4/6) = 0.91$

Expanding the summation in the IG formula:

$IG(S, \text{Fever}) = \text{Entropy}(S) - (|S_{YES}| / |S|) * \text{Entropy}(S_{YES}) - (|S_{NO}| / |S|) * \text{Entropy}(S_{NO})$

$\therefore IG(S, \text{Fever}) = 0.99 - (8/14) * 0.81 - (6/14) * 0.91 = 0.13$

- calculate the **IG** for the features **"Cough"** and **"Breathing issues".**
- **IG(S, Cough) = 0.04**
  IG(S, BreathingIssues) = 0.40



- Next, from the remaining two unused features, namely, **Fever** and **Cough**, we decide which one is the best for the left branch of **Breathing Issues**.
  Since the left branch of **Breathing Issues** denotes **YES,** we will work with the subset of the original data i.e the set of rows having **YES** as the value in the Breathing Issues column. These **8 rows** are shown below:

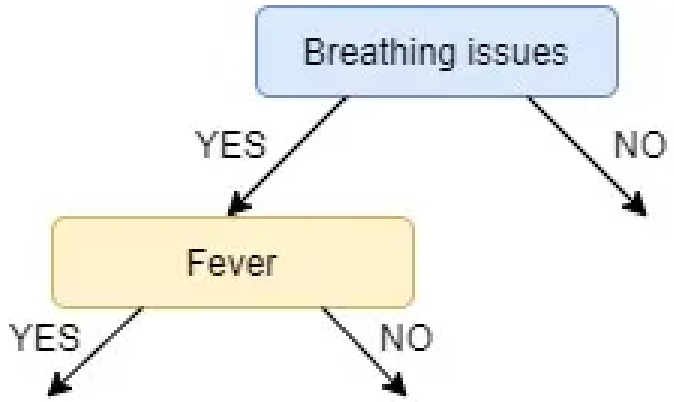
- **Fever | Cough | Breathing issues | Infected |**

```
+-------+-------+-----------------+----------+
|YES |YES |YES |YES |
+-------+-------+-----------------+----------+
|YES | NO |YES |YES |
+-------+-------+-----------------+----------+
|YES |YES |YES |YES |
+-------+-------+-----------------+----------+
|YES | NO |YES |YES |
+-------+-------+-----------------+----------+
|YES | NO |YES |YES |
+-------+-------+-----------------+----------+
| NO |YES |YES |YES |
+-------+-------+-----------------+----------+
| NO |YES |YES |YES |
+-------+-------+-----------------+----------+
| NO |YES |YES | NO
```

- Next, we calculate the IG for the features Fever and Cough using the subset **S**BY (**S**et **Breathing** Issues **Y**es)

- *For **IG** calculation the Entropy will be calculated from the subset **S**BY and not the original dataset **S.***

- **IG(SBY, Fever) = 0.20**
  IG(SBY, Cough) = 0.09



- There are no more unused features, so we stop here and jump to the final step of creating the leaf nodes.

- For the left leaf node of Fever, we see the subset of rows from the original data set that has **Breathing Issues** and **Fever** both values as **YES**.

- **ever | Cough | Breathing issues | Infected |**

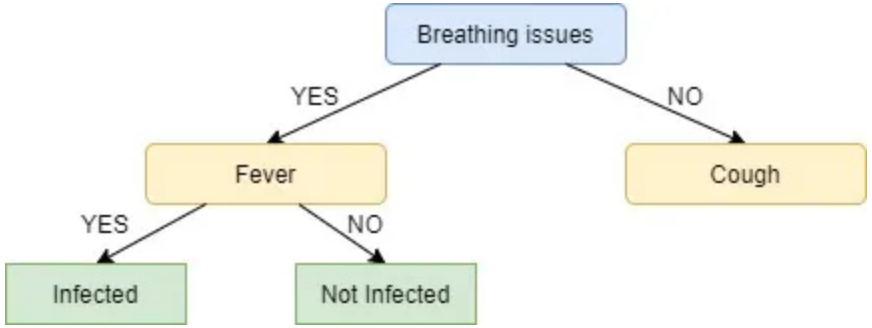
```
+-------+-------+----------------+----------+
|YES |YES |YES |YES |
+-------+-------+----------------+----------+
|YES | NO |YES |YES |
+-------+-------+----------------+----------+
|YES |YES |YES |YES |
+-------+-------+----------------+----------+
|YES | NO |YES |YES |
+-------+-------+----------------+----------+
|YES | NO |YES |YES |
+-------+-------+----------------+----------
```

- Similarly, for the right node of Fever we see the subset of rows from the original data set that have **Breathing Issues** value as **YES** and **Fever** as **NO**.

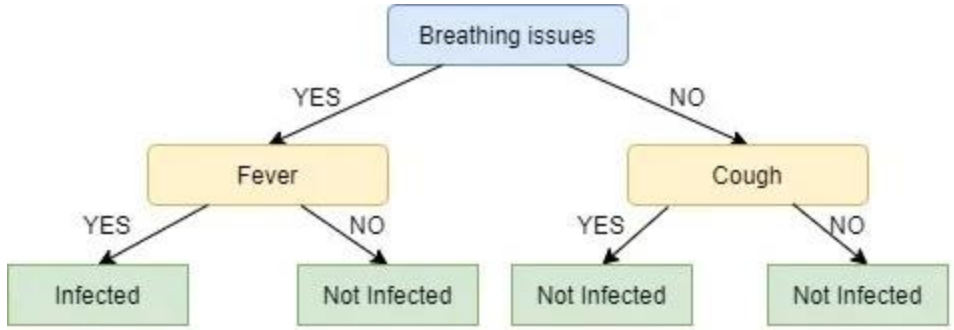
- **Fever | Cough | Breathing issues | Infected |**

```
+-------+-------+----------------+----------+
| NO |YES |YES |YES |
+-------+-------+----------------+----------+
| NO |YES |YES | NO |
+-------+-------+----------------+----------+
| NO |YES |YES | NO
```

- Here not all but **most** of the **values** are **NO,** hence **NO** or **Not Infected** becomes our **right leaf node.** Our tree, now, looks like this:
-



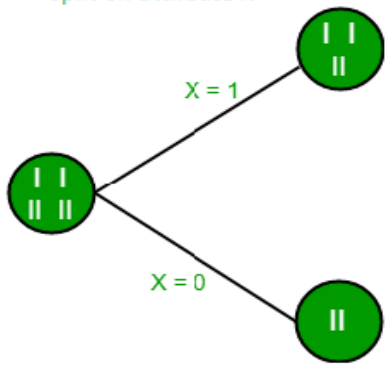
- We repeat the same process for the node **Cough**, however here both left and right leaves turn out to be the same i.e. **NO** or **Not Infected** as shown below:



- Now, let us draw a Decision Tree for the following data using Information gain. **Training set: 3 features and 2 classes**

X	Y	Z	C
I	I	I	I
I	I	0	I
0	0	I	II
I	0	0	II

Split on attribute X



$E_{parent} = 1$

$GAIN = 1 - (3/4)(0.9184) - (1/4)(0) = 0.3112$

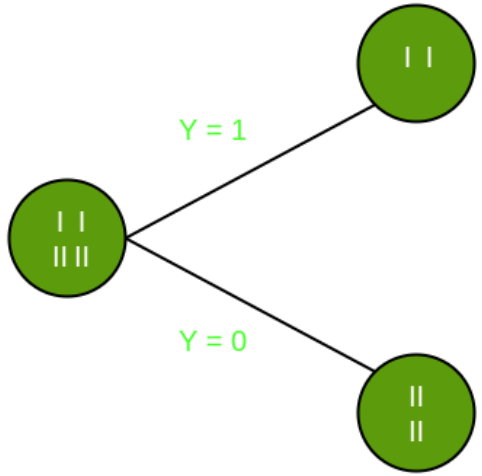
$E_{child1} = -(1/3)\log_2(1/3) - (2/3)\log_2(2/3)$

$= 0.5284 + 0.39$

$= 0.9184$

$E_{child2} = 0$

Split an attribute Y



$E_{parent} = 1$

$GAIN = 1 - (1/2)(0) - (1/2)(0) = 1$

$E_{child1} = 0$

$E_{child2} = 0$

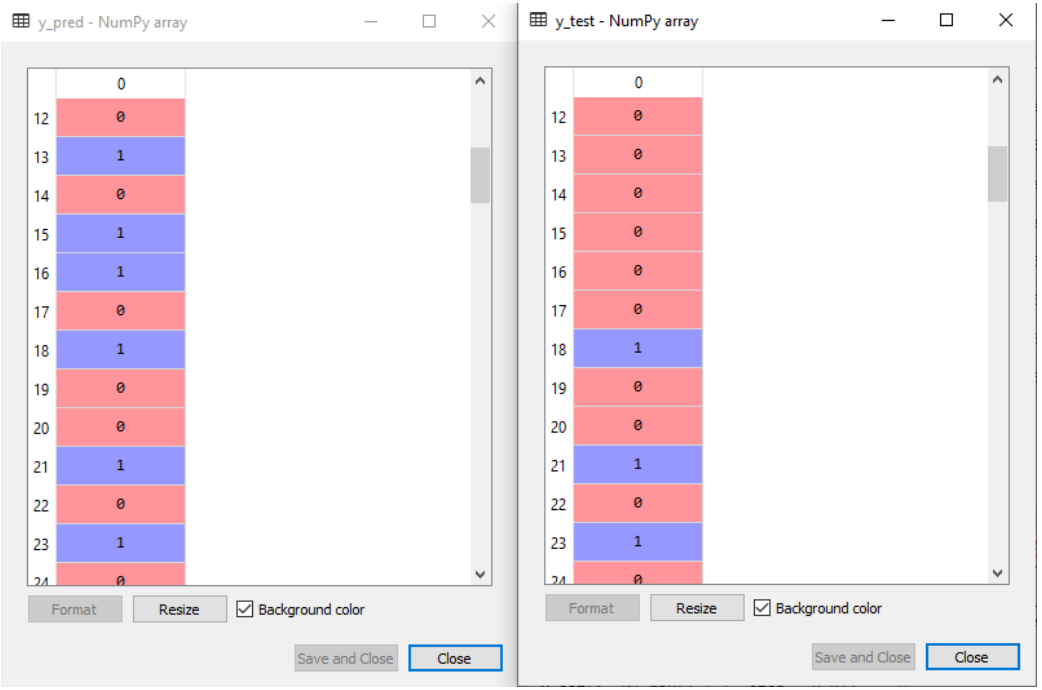
```
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
#importing datasets
data_set= pd.read_csv('user_data.csv')
#Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values
# Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)
#feature Scaling
from sklearn.preprocessing import StandardScaler
st_x= StandardScaler()
x_train= st_x.fit_transform(x_train)
x_test= st_x.transform(x_test)
```

data_set - DataFrame

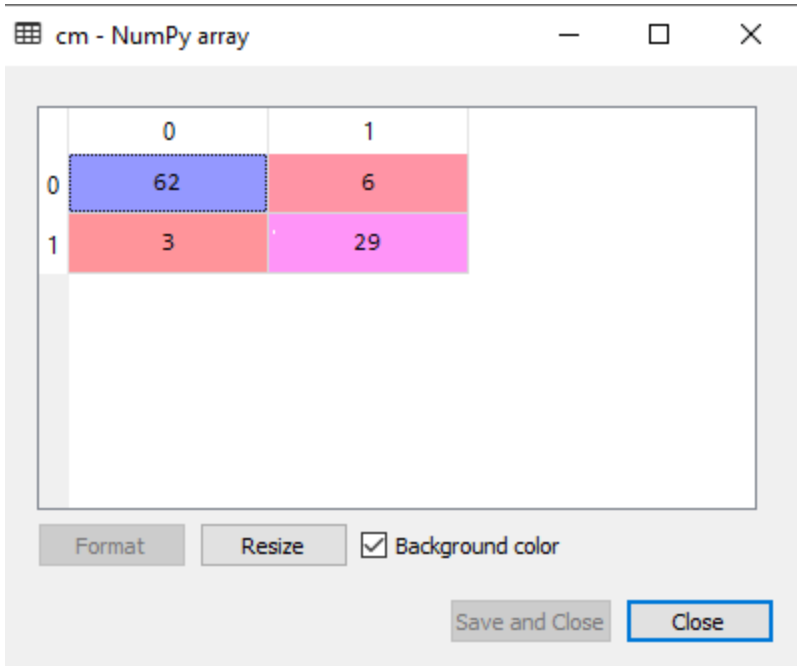
Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0
14	15628972	Male	18	82000	0
15	15697686	Male	29	80000	0

Format Resize ☑ Background color ☑ Column min/max Save and Close Close

- Fitting a Decision-Tree algorithm to the Training set
- #Fitting Decision Tree classifier to the training s et
- From sklearn.tree **import** DecisionTreeClassifi er
- classifier= DecisionTreeClassifier(criterion='ent ropy', random_state=0)
- classifier.fit(x_train, y_train)
- Predicting the test result
- #Predicting the test set result
- y_pred= classifier.predict(x_test)

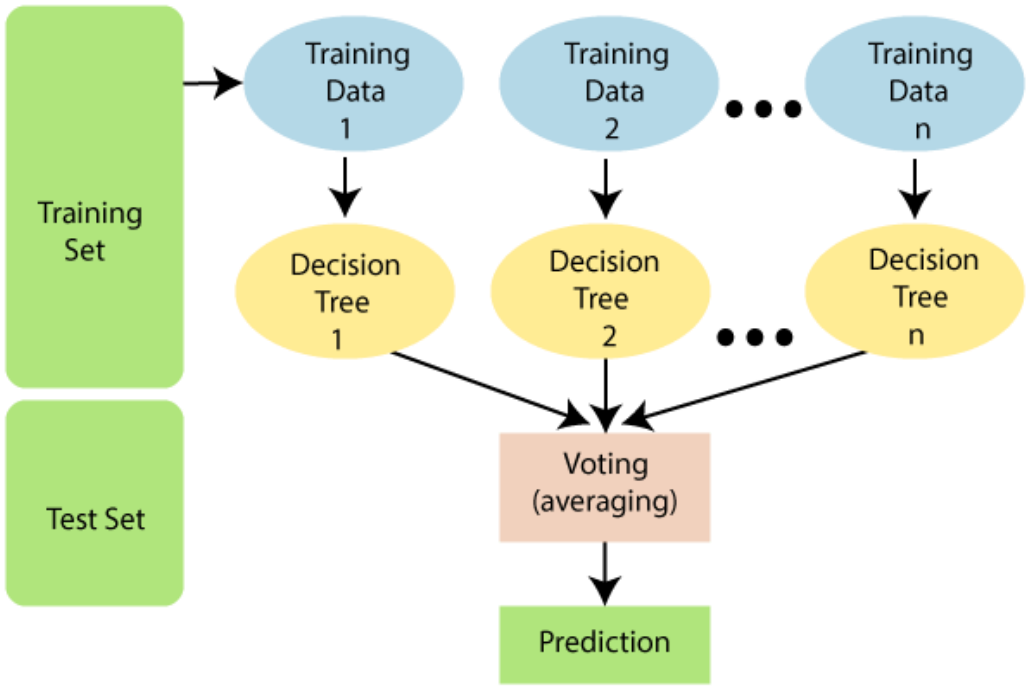


- Test accuracy of the result
- #Creating the Confusion matrix
- from sklearn.metrics **import** confusion_matrix
- cm= confusion_matrix(y_test, y_pred)



Random Forest

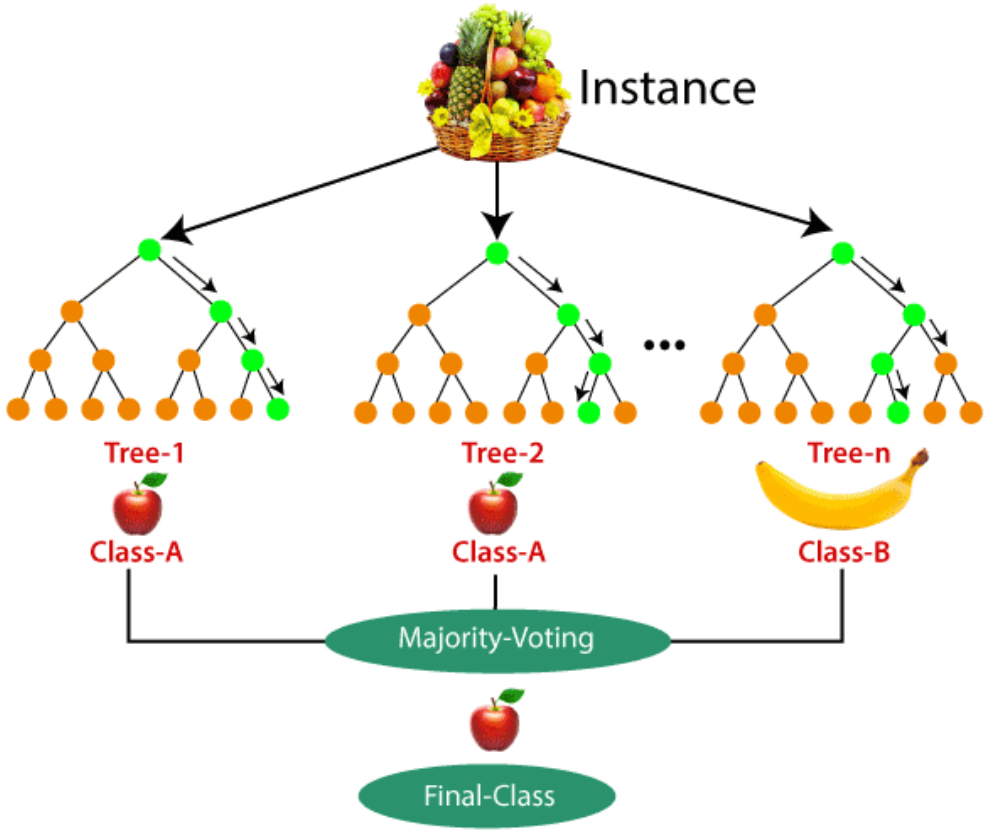
- It is based on the concept of **ensemble learning,** which is a process of *combining multiple classifiers to solve a complex problem.*
- ***Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.***
- **The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.**



- Assumptions for Random Forest
- There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.
- The predictions from each tree must have very low correlations.

- Why use Random Forest?
- It takes less training time as compared to other algorithms.
- It predicts output with high accuracy, even for the large dataset it runs efficiently.
- It can also maintain accuracy when a large proportion of data is missing.

- How does Random Forest algorithm work?
- **Step-1:** Select random K data points from the training set.
- **Step-2:** Build the decision trees associated with the selected data points (Subsets).
- **Step-3:** Choose the number N for decision trees that you want to build.
- **Step-4:** Repeat Step 1 & 2.
- **Step-5:** For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.



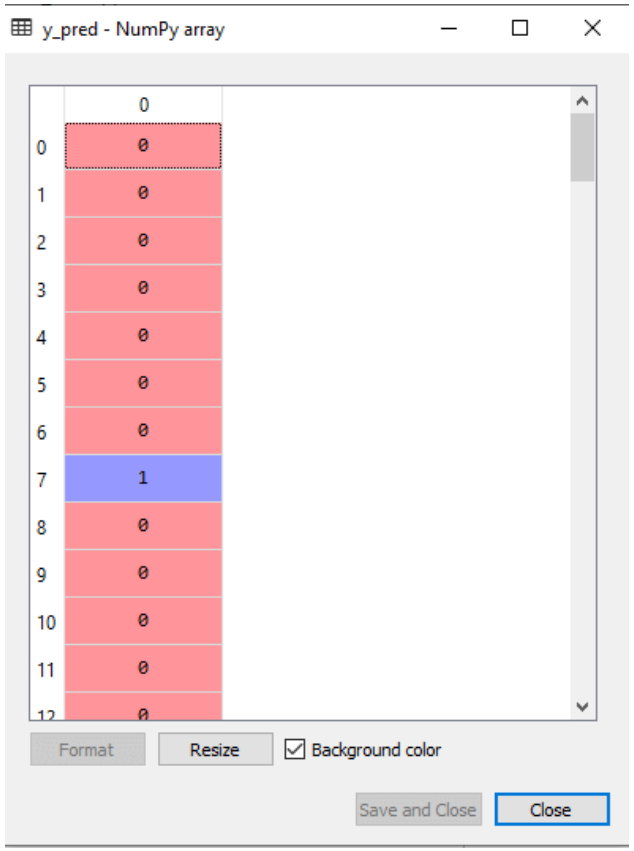
- Data Pre-Processing Step:
- # importing libraries
- **import** numpy as nm
- **import** matplotlib.pyplot as mtp
- **import** pandas as pd
- #importing datasets
- data_set= pd.read_csv('user_data.csv')
- #Extracting Independent and dependent Variable
- x= data_set.iloc[:, [2,3]].values
- y= data_set.iloc[:, 4].values
- # Splitting the dataset into training and test set.
- from sklearn.model_selection **import** train_test_split
- x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)
- #feature Scaling
- from sklearn.preprocessing **import** StandardScaler
- st_x= StandardScaler()
- x_train= st_x.fit_transform(x_train)
- x_test= st_x.transform(x_test)

data_set - DataFrame

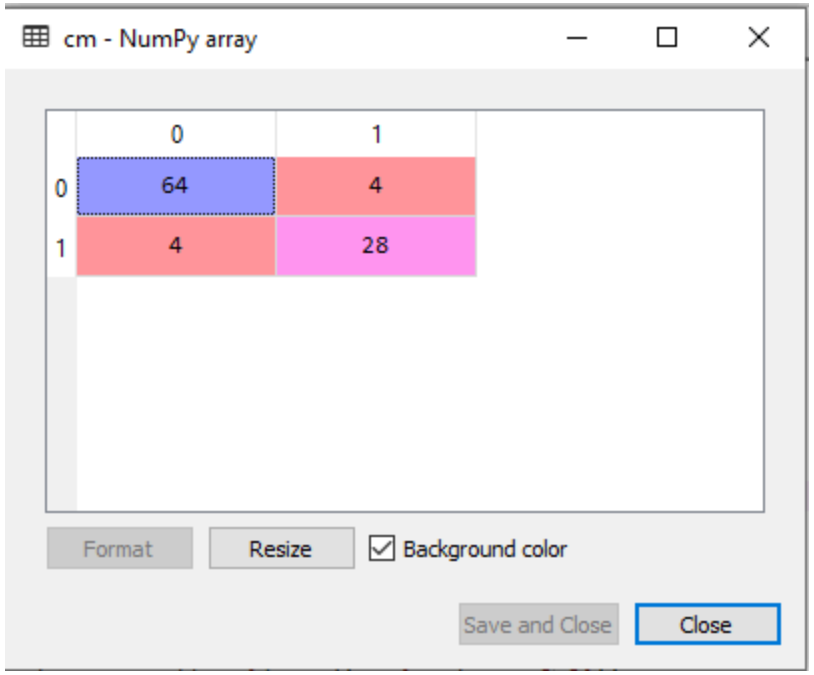
Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0

Format Resize ☑ Background color ☐ Column min/max Save and Close Close

- Fitting the Random Forest algorithm to the training set:
- #Fitting Decision Tree classifier to the training s et
- from sklearn.ensemble **import** RandomForest Classifier
- classifier= RandomForestClassifier(n_estimator s= 10, criterion="entropy")
- classifier.fit(x_train, y_train)
- 3. Predicting the Test Set result
- #Predicting the test set result
- y_pred= classifier.predict(x_test)



- Creating the Confusion Matrix
- #Creating the Confusion matrix
- from sklearn.metrics **import** confusion_matrix
- cm= confusion_matrix(y_test, y_pred)

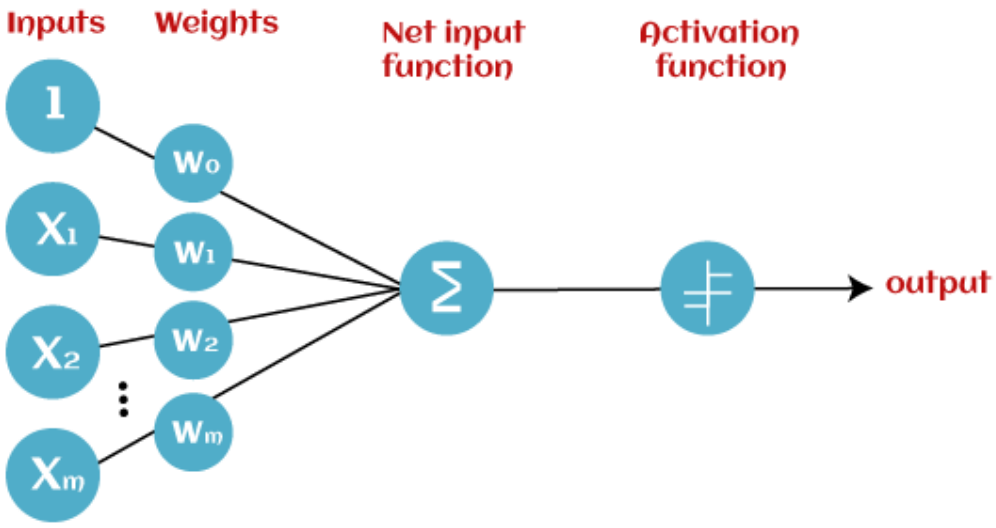


Perceptron in Machine Learning

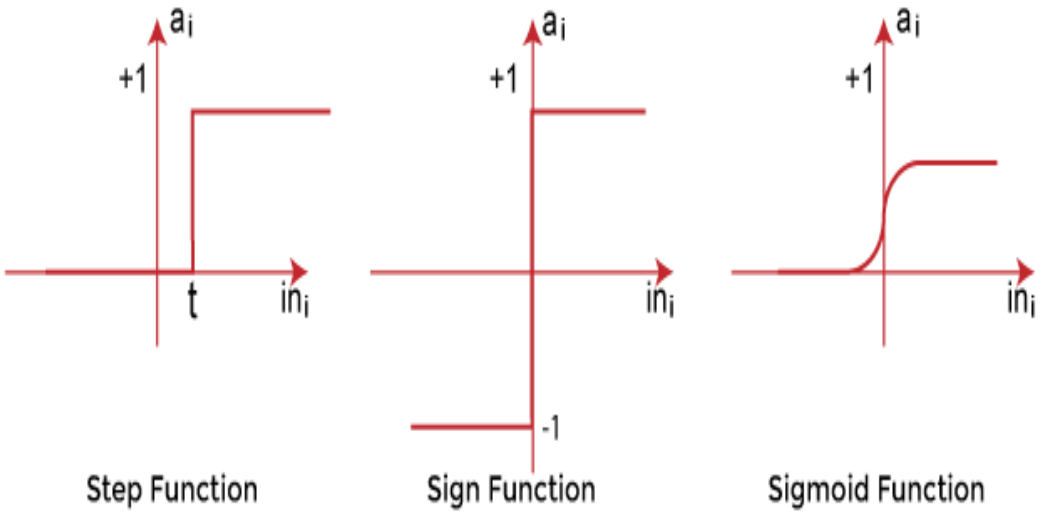
- ***Perceptron is a building block of an Artificial Neural Network.***
- **Mr. Frank Rosenblatt** invented the Perceptron for performing certain calculations to detect input data capabilities or business intelligence.
- Perceptron is a linear Machine Learning algorithm used for supervised learning for various binary classifiers

- What is the Perceptron model in Machine Learning?
- ***Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence***.
- it as a single-layer neural network with four main parameters, i.e., **input values, weights and Bias, net sum, and an activation function.**

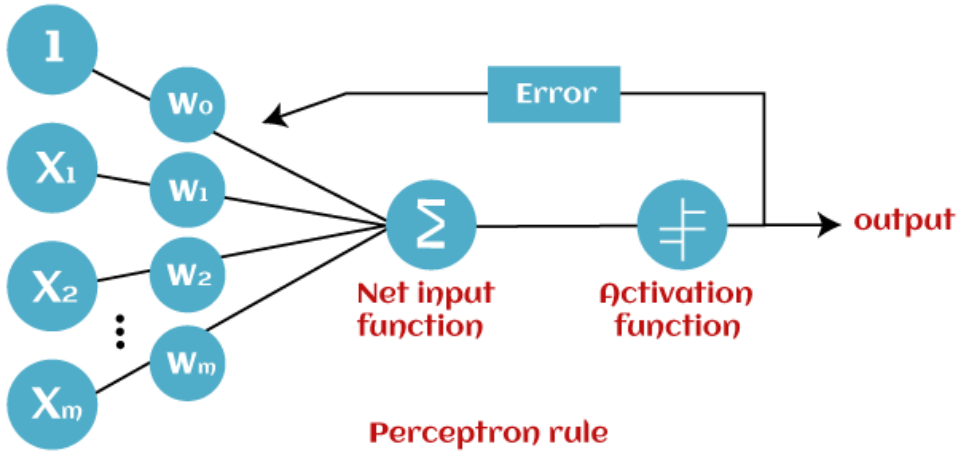
- Basic Components of Perceptron
- **Input Nodes or Input Layer:**
- accepts the initial data into the system
- **Wight and Bias:**
- Weight parameter represents the strength of the connection between units.
- Bias can be considered as the line of intercept in a linear equation.
- **Activation Function:**
- final and important components that help to determine whether the neuron will fire or not.



- Types of Activation functions:
- Sign function
- Step function, and
- Sigmoid function



- How does Perceptron work?
- **Step-1**
- multiply all input values with corresponding weight values and then add them to determine the weighted sum
- ∑wi*xi = x1*w1 + x2*w2 +…wn*xn
- Add a special term called **bias 'b'** to this weighted sum to improve the model's performance.
- **∑wi*xi + b**



- **Step-2**
- an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:
- **$Y = f(\sum wi*xi + b)$**

- Types of Perceptron Models
- **Single-Layer Perceptron** is a type of perceptron is limited to learning linearly separable patterns. It is effective for tasks where the data can be divided into distinct categories through a straight line.
- **Multi-Layer Perceptron** possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

- **Example: Perceptron in Action**
- Let's take a simple example of classifying whether a given fruit is an apple or not based on two inputs: its weight (in grams) and its color (on a scale of 0 to 1, where 1 means red). The perceptron receives these inputs, multiplies them by their weights, adds a bias, and applies the activation function to decide whether the fruit is an apple or not.
- Input 1 (Weight): 150 grams
- Input 2 (Color): 0.9 (since the fruit is mostly red)
- Weights: [0.5, 1.0]
- Bias: 1.5
- The perceptron's weighted sum would be:
- $(150*0.5)+(0.9*1.0)+1.5=76.4(150*0.5)+(0.9*1.0)+1.5=76.4$
- *Let's assume the activation function uses a threshold of 75. Since 76.4 > 75, the perceptron classifies the fruit as an apple (output = 1).*

- **Binary Classification on a Linearly Separable Dataset**
- Import Libraries
- Generate Dataset using **make_blobs()**
- Train-Test Split with **train_test_split()**
- Scale Features using **StandardScaler()**
- Initialize Perceptron with appropriate input size
- Train the Model with **fit()** over 100 epochs
- Predict on test data and evaluate accuracy by comparing predictions with actual labels
- Visualize Results using a scatter plot

- # Import the necessary library
- import numpy as np
- from sklearn.datasets import make_blobs
- import matplotlib.pyplot as plt
- from sklearn.model_selection import train_test_split
- from sklearn.preprocessing import StandardScaler

- # Generate a linearly separable dataset with two classes
- X, y = make_blobs(n_samples=1000,
- n_features=2,
- centers=2,
- cluster_std=3,
- random_state=23)
- # Split the dataset into training and testing sets
- X_train, X_test, y_train, y_test = train_test_split(X,
- y,
  - test_size=0.2,
- random_state=23,
- shuffle=True
-)

```
# Scale the input features to have zero mean and unit variance
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Set the random seed legacy
np.random.seed(23)
# Initialize the Perceptron with the appropriate number of inputs
perceptron = Perceptron(num_inputs=X_train.shape[1])

# Train the Perceptron on the training data
perceptron.fit(X_train, y_train, num_epochs=100)
```

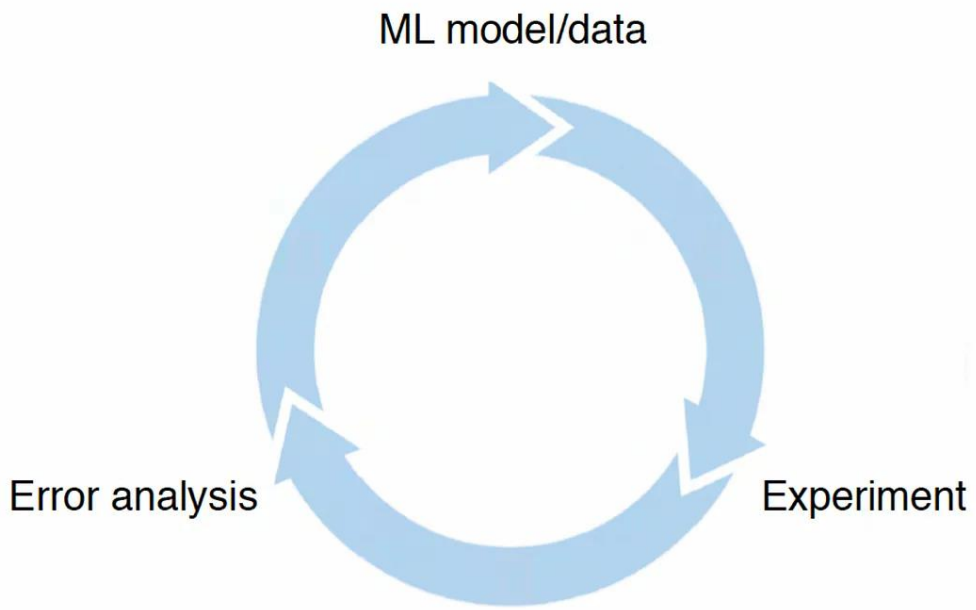
```
# Prediction
pred = perceptron.predict(X_test)

# Test the accuracy of the trained Perceptron on the testing data
accuracy = np.mean(pred != y_test)
print("Accuracy:", accuracy)

# Plot the dataset
plt.scatter(X_test[:, 0], X_test[:, 1], c=pred)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Error Analysis

- Error analysis is the process of identifying and understanding the sources of errors or inaccuracies in a machine learning model.
- It is a critical step in the model development process, as it helps data scientists to understand the strengths and weaknesses of their model, and to identify areas for improvement.



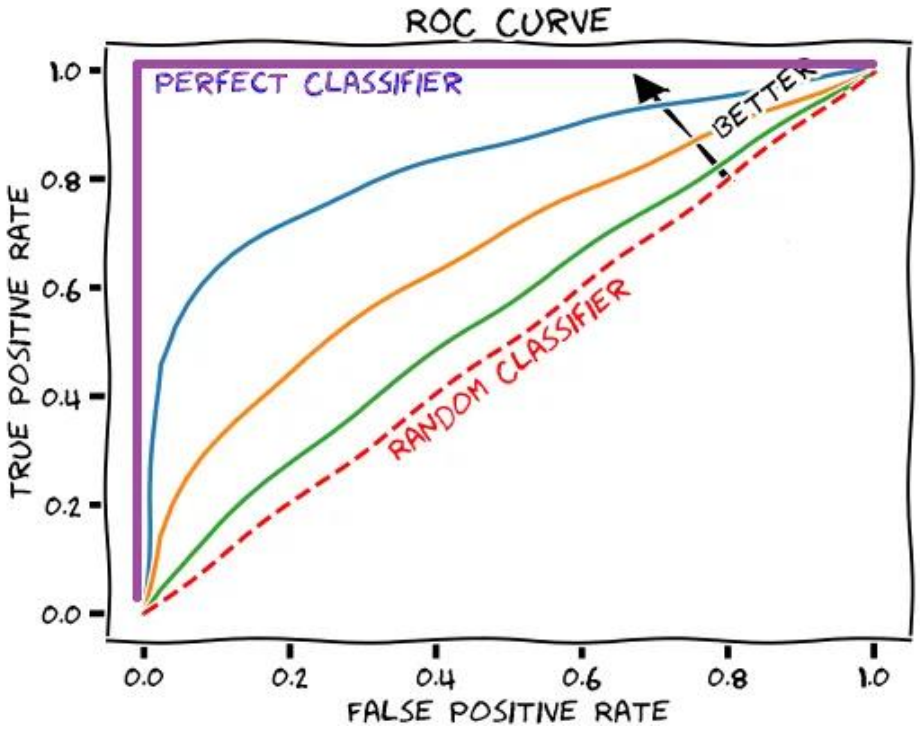
Error Analysis

- There are several common techniques used in error analysis.
- **Confusion matrix**: A confusion matrix is a table that is used to define the performance of a classification algorithm. It helps to identify the number of correct and incorrect predictions made by the model.
-

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	**Sensitivity** $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

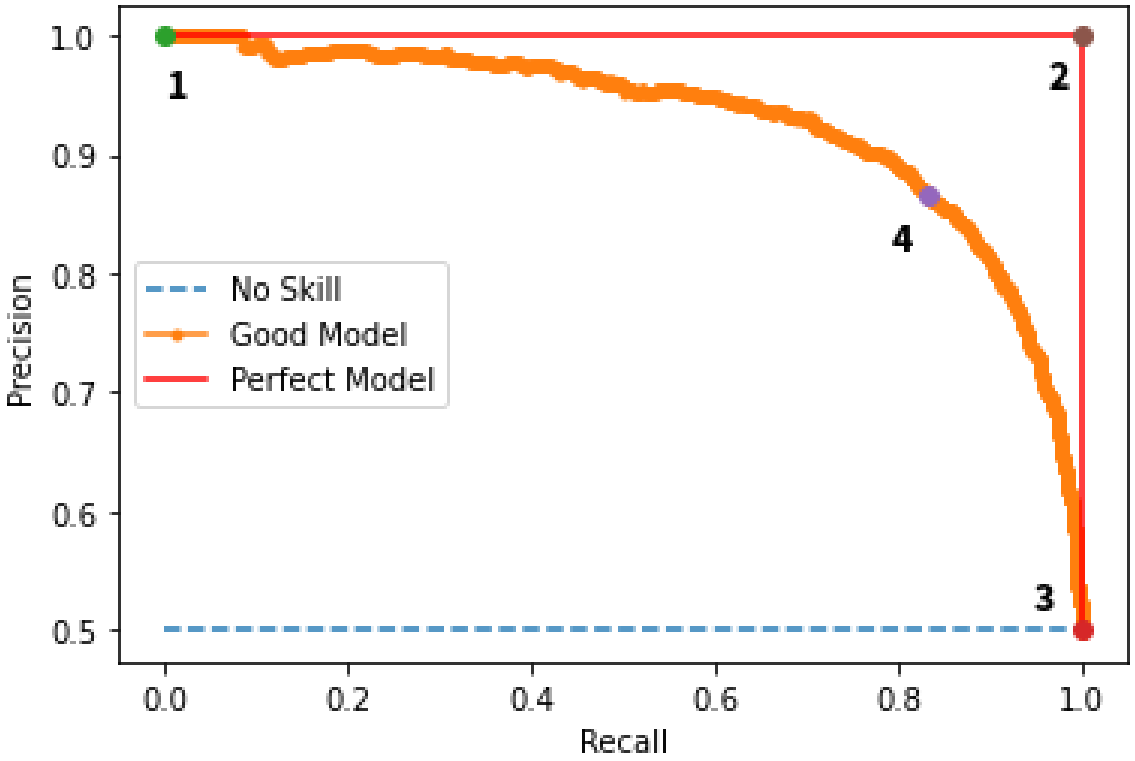
Error Analysis

- **ROC Curve:** Receiver Operating Characteristic (ROC) curve is a graphical representation of the performance of a binary classifier system as the discrimination threshold is varied.
- It is a plot of the true positive rate against the false positive rate.



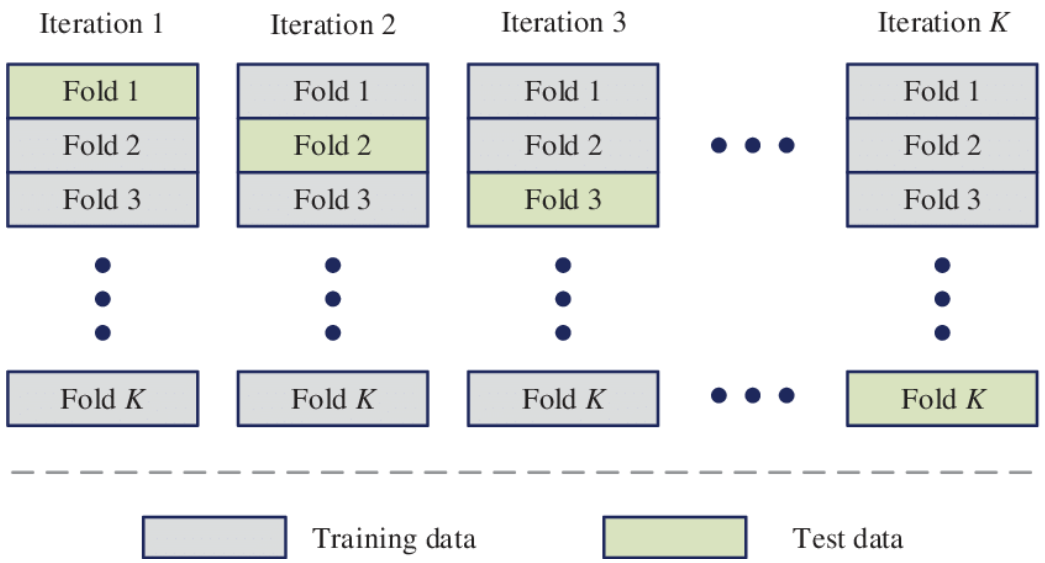
Error Analysis

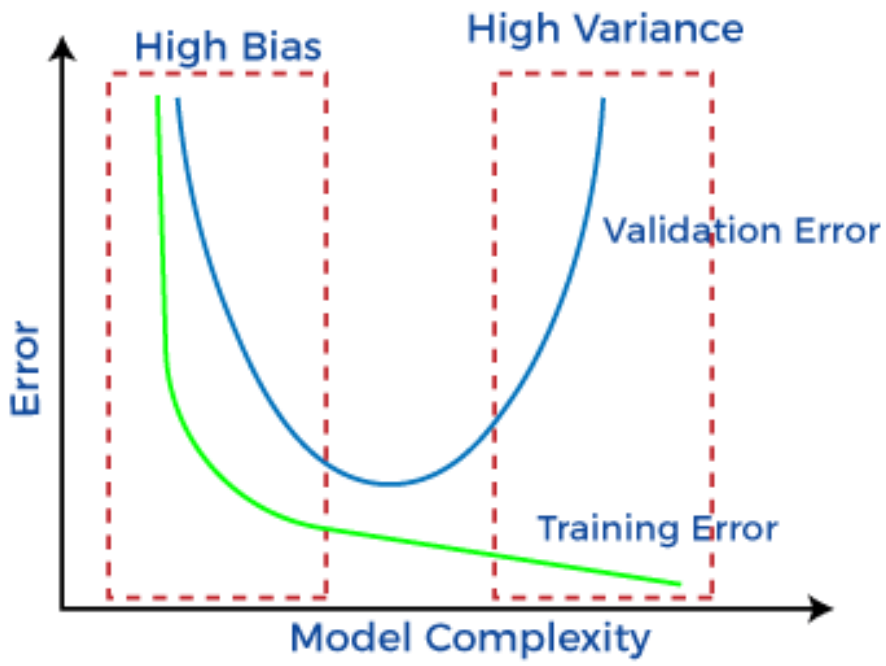
- **Precision-recall curve**: A precision-recall curve is a graphical representation of the trade-off between the precision and recall of a classifier.
- **X-axis (Recall)**: Measures how many actual positives were correctly identified
- **Y-axis (Precision)**: Measures the proportion of predicted positives that are correct.



Error Analysis

- **K-fold cross validation**: It is a resampling method used to evaluate machine learning models on a limited data sample.



Error Analysis

- **Bias-variance tradeoff**: The bias-variance tradeoff is the tradeoff between the model's ability to fit the training data well and its ability to generalize to new data.
- **High Bias Region**
- **Low complexity models (e.g., linear regression on non-linear data). Underfitting:** Model is too simple, failing to capture patterns. Both **training and validation errors are high**.
- High complexity models (e.g., deep neural networks with too many parameters) Overfitting: Model memorizes training data but generalizes poorly. Training error is very low, but validation error increases